

标准机械臂批量逆运动学求解的 CUDA 并行映射与性能边界分析

摘要

机械臂批量逆运动学（IK）求解具有小矩阵、多迭代和强控制流依赖等特征，其 GPU 加速效果取决于具体并行映射和硬件资源利用方式，而非简单代码移植。6×6 小矩阵求解、迭代控制流、自适应阻尼和大量独立目标的调度开销，决定了这一问题必须做硬件感知的底层映射。本文以官方 UR10 为主实验对象，统一模型、`tool0`、目标位姿资产、初始种子和误差阈值，构建了可复现的批量 IK benchmark。

本文的核心方法是将每个目标映射为 1 个 CUDA block，并在 block 内用 128 threads 分工完成 FK、数值 Jacobian、阻尼正规方程构造与寄存器级 6×6 LDLT 求解。为保证结果可比较，本文只保留标准 UR10 批量 IK 查询，不引入路径规划、碰撞检测、轨迹回放等上层环节。本文进一步统一了位姿误差定义、DLS 阻尼项和共享内存数据布局，保证实验结果具有一致的模型与数值口径。

实验上，本文用 B0-B6 消融、cuRobo 统一口径比较、三档组合阈值扫描、N=100→10000 全量程固定步长扫描（12 个 N 值）、Nsight Compute profiling 和 7DOF Panda 正确性验证构成证据链。结果表明：常量内存与 `PaddedMat6x8` 只带来有限收益，自适应阻尼决定收敛率，混合精度有效提高吞吐；CUDA B5 在全量程上吞吐稳定（148k–174k, ±8%），GPU 时间与批量规模严格线性（R²>0.999），而 cuRobo 在本文统一 benchmark 配置下于 4/12 个 N 值上出现求解时间从 ~32ms 跳变至 ~230ms 的批量敏感振荡——进一步源码检查与时间特征分析表明，该退化更可能来自高层框架的 workspace 管理、内存分配或同步开销，而非 IK 迭代计算本身，其精确机制仍需 Nsight Systems 与 CUDA API trace 进一步确认；CUDA B5 因裸 `cudaMalloc` 一次性分配和单 kernel 全迭代封装而不依赖第三方内存管理器，对此类开销具有结构免疫性；在 cuRobo 正常模式下，其在 N≥6000 时吞吐可反超 CUDA B5（最高 1.43×），但退化模式使其吞吐骤降至 18k–42k，性能可预测性不足；NCU 结果说明该 kernel 主要受计算吞吐与寄存器压力约束，而非 DRAM 带宽约束；Panda 结果只用于正确性验证，不作为正式性能对比。

关键词

CUDA；inverse kinematics；UR10；standardized benchmark；Nsight Compute；small-matrix solver

论文结构

- 第 1 章 引言
- 第 2 章 相关工作
- 第 3 章 批量逆运动学问题建模
- 第 4 章 CUDA 并行求解器设计
- 第 5 章 实验设计与评价方法
- 第 6 章 实验结果与性能分析
- 第 7 章 扩展性验证
- 第 8 章 讨论
- 第 9 章 结论

1 引言

本文研究的是标准机械臂批量逆运动学的 GPU 实现，而不是路径规划、轨迹回放或碰撞感知 IK。以官方 UR10 为主实验对象，统一模型、`tool0`、目标位姿资产、初始种子和误差阈值，本文构建了一个可复现的批量 IK benchmark，并据此评估 CUDA 并行映射、数值稳定性和硬件资源利用。

这一问题的关键不在于“把代码搬到 GPU”，而在于“如何把一个固定规模的小矩阵迭代过程稳定映射到 GPU”。每个目标都要重复执行 FK、位姿误差、数值 Jacobian、阻尼正规方程和 6×6 线性系统求解；这些步骤对寄存器、共享内存和 launch 开销都很敏感。因此，本文把问题范围收束为标准 UR10 批量 IK 查询，不再引入轨迹连续性、碰撞检测或其他上层规划环节。

研究背景

批量 IK 的典型应用是离线采样、抓取候选生成和标定辅助等任务。这类任务的特点是单目标代价不高，但目标数很多，因此真正决定体验的是吞吐量和失败率，而不是单次求解是否“足够优雅”。对于这类问题，GPU 的价值在于把大量独立目标并行展开，而不是引入更复杂的外层搜索。

问题定义

本文固定同一 UR10 模型、同一 `tool0` 定义、同一批可达目标位姿和同一组初始种子，在统一位置/姿态误差阈值下比较不同求解器的批量 IK 性能。这样定义的好处是边界清楚：它只衡量求解器本身对独立目标的处理能力，不混入路径连续性、避障约束或轨迹优化的额外因素。

技术挑战

挑战主要来自三个层面。第一， 6×6 小矩阵虽然规模小，但需要在每次迭代中反复求解，通用线性代数库往往难以充分利用其固定结构。第二，DLS 的收敛性高度依赖阻尼、步长和初始种子。第三，GPU 性能不仅由算术吞吐决定，还受寄存器压力、共享内存访问和 kernel launch 调度的共同影响。

方法概述

本文采用 1 block/target、128 threads/block 的并行映射，把一个目标的全部 DLS 迭代压缩到单次 kernel 中完成。block 内按 warp 分工完成 FK/误差、数值 Jacobian、阻尼正规矩阵/梯度和步长更新；共享内存用 `PaddedMat6x8`，线性系统用寄存器级 LDLT，主体计算采用 FP32，关键求解路径保留 FP64。围绕这个主线，本文再通过消融、cuRobo 比较、Nsight Compute profiling 和 Panda 正确性验证建立证据链。

主要贡献

本文的贡献只有三项：

1. 构建标准 UR10 批量 IK benchmark，统一模型、`tool0`、种子、阈值与时间口径。
2. 提出面向 6DOF 批量 IK 的 CUDA 并行映射和寄存器级 6×6 LDLT 实现。
3. 通过 B0-B6 消融、cuRobo 统一口径比较、Nsight Compute 证据链和 7DOF Panda 正确性验证，说明性能边界与扩展性。

2 相关工作

DLS / LM / KDL

阻尼最小二乘法（DLS）是机械臂 IK 中最常用的迭代方法之一，Levenberg-Marquardt（LM）则是在此基础上引入自适应阻尼调度的常见变体。它们的共同优势是问题形式清楚、可直接实现、每步代价可分析；共同限制

是当目标数增大时，CPU 端的串行迭代会快速累积时间。Orocos KDL 提供了成熟的 FK/IK 工具链，适合作为保守的 CPU 参考基线，但不是面向大批量独立目标的高吞吐方案。

cuRobo / PyRoki

cuRobo 和 PyRoki 代表了两类主流 GPU/加速求解思路。前者强调多粒子并行和工程集成，后者强调 JAX 生态下的可组合性与研究灵活性。二者都说明了一个事实：GPU IK 的性能不仅取决于数学算法，还取决于框架层级、launch 次数、批处理策略和精度路径。本文不追求在这些系统之上再做一个“更通用”的上层框架，而是把问题收缩到标准 UR10 批量 IK 的底层实现。

小矩阵求解

本文的核心数值核是固定规模的 6×6 正定线性系统。对这类小矩阵而言，通用大矩阵库的调度粒度通常过粗，寄存器级展开或轻量共享内存布局往往更合适。与其讨论大规模线性代数的吞吐上限，不如先解决一个更具体的问题：如何把 6×6 LDLT 稳定、低开销地嵌入到 DLS 迭代里。

本文定位

本文工作的定位是：在统一 benchmark 下，把标准 UR10 批量 IK 的数学形式、CUDA 并行映射、数值稳定性和硬件 profiling 证据连成一条线。它不是一个路径规划系统，也不是一个碰撞感知 IK 系统；它关心的是标准批量 IK 查询本身的性能边界，以及这条边界在 6DOF 和 7DOF 上如何变化。

3 批量逆运动学问题建模

本章建立批量逆运动学求解的完整数学模型。从 UR10 串联机械臂的前向运动学链式公式出发，经位姿误差的 $SE(3)$ 定义、阻尼最小二乘（DLS）迭代格式、 LDL^T 小矩阵线性系统求解，最终将问题归结为批量任务定义与 CUDA 并行映射框架。本章的数学推导直接服务于第四章的 CUDA kernel 设计——每个数学对象的维度、运算量 and 数据依赖性均在此明确，为后续的 Warp 分工、共享内存布局和寄存器分配提供形式化依据。

UR10 串联机械臂运动学模型

关节变量与前向运动学链式乘积

本文主实验平台为 Universal Robots UR10——6 自由度全旋转关节串联机械臂。其关节构型由 6 维关节角向量描述：

$$\mathbf{q} = [q_1, q_2, q_3, q_4, q_5, q_6]^T \in \mathbb{R}^6$$

其中 q_i 为第 i 个旋转关节的角位移（单位：rad），受限於 URDF 中声明的关节限位 $[q_{i,\min}, q_{i,\max}]$ 。

末端执行器（tool0）在基座坐标系下的位姿由齐次变换矩阵 $\mathbf{T}_{EE}(\mathbf{q}) \in SE(3)$ 表示：

$$\mathbf{T}_{EE}(\mathbf{q}) = \begin{bmatrix} \mathbf{R}(\mathbf{q}) & \mathbf{p}(\mathbf{q}) \\ \mathbf{0}^T & 1 \end{bmatrix}$$

其中 $\mathbf{R}(\mathbf{q}) \in SO(3)$ 为 3×3 旋转矩阵， $\mathbf{p}(\mathbf{q}) \in \mathbb{R}^3$ 为位置向量。该位姿由 $n = 6$ 个关节变换矩阵与末端工具变换的链式乘积给出：

$$\mathbf{T}_{EE}(\mathbf{q}) = \prod_{i=1}^6 (\mathbf{T}_i^0 \cdot \mathbf{R}(\mathbf{a}_i, q_i)) \cdot \mathbf{T}_{\text{tool}}$$

其中 $\mathbf{T}_i^0 \in SE(3)$ 为第 i 个关节的固定原点变换（由URDF中关节origin的xyz/rpy确定）， $\mathbf{a}_i \in \mathbb{R}^3$ 为关节旋转轴单位向量（ $\|\mathbf{a}_i\| = 1$ ）， $\mathbf{R}(\mathbf{a}_i, q_i)$ 为绕轴 $\mathbf{a}_i$ 旋转 q_i 的Rodrigues旋转矩阵， $\mathbf{T}_{\text{tool}}$ 为wrist_3坐标系至tool0的固定工具变换。

Rodrigues旋转公式与反对称矩阵

旋转矩阵 $\mathbf{R}(\mathbf{a}, q) \in SO(3)$ 由Rodrigues旋转公式给出：

$$\mathbf{R}(\mathbf{a}, q) = \mathbf{I} + \sin(q)[\mathbf{a}]_{\times} + (1 - \cos(q))[\mathbf{a}]_{\times}^2$$

其中 $[\mathbf{a}]_{\times}$ 为 $\mathbf{a} = [a_x, a_y, a_z]^T$ 的反对称矩阵：

$$\begin{bmatrix} 0 & -a_z & a_y \\ a_z & 0 & -a_x \\ -a_y & a_x & 0 \end{bmatrix}$$

上式将旋转矩阵的计算分解为 $\sin(q)$ 与 $\cos(q)$ 的标量求值（各1次三角函数）及 3×3 矩阵的线性组合（约12次乘法运算），避免了全矩阵指数映射的高昂代价。在GPU上， $\sin$ 与 $\cos$ 经专用特殊函数单元（Special Function Unit, SFU）流水线执行，可与其他算术指令实现部分重叠。

固定变换折叠与标准化一致性

上式的表述隐含了一项关键的工程前提：链式乘积中**所有**固定变换均已被正确折叠。在实际URDF解析中，相邻主动关节之间存在若干被动固定变换——例如 `\texttt{base_link} \rightarrow \texttt{base_link_inertia}` 的惯量框架偏移、`\texttt{wrist_3} \rightarrow \texttt{tool0}` 的工具坐标系变换，以及各连杆间的视觉/碰撞框架变换。若其中任一固定变换在FK计算中被遗漏，CPU与GPU求解器可能在**同一错误链路上**保持内部自治——即CPU FK与GPU FK输出一致——但与官方URDF模型的真值位姿产生系统性偏差。此类偏差在位置误差上可能达到厘米量级、在姿态误差上可达到度量级，直接破坏收敛判据的可信性。

在标准化实现中，所有固定变换均由URDF自动解析并导出为CUDA常量参数，避免人工硬编码导致的模型不一致。本文在标准化重构中将所有固定变换统一由 `robot\_model.py` 从平铺URDF中自动解析并导出为CUDA常量内存参数，从模型层面消除了这一偏差源。

FK运算复杂度

单次FK计算的运算量由三部分构成：（1）6次Rodrigues旋转公式求值，每次含2次三角函数与约12次乘法；（2）5次 4×4 齐次矩阵乘法（6个关节变换与1个工具变换的链式组合），每次 $4 \times 4 \times 4 = 64$ 次乘法；（3）1次末端工具变换乘法。总标量运算量约为：

$$\text{FLOP}_{\text{FK}} = 6 \times (2 \text{ trig} + 15) + 6 \times 64 \approx 538$$

该运算量的工程意义在于：单个FK调用的计算成本约相当于1--2次 6×6 矩阵乘法，意味着在DLS迭代中数值雅可比所需的12次FK调用（见下文）是每轮迭代的主要计算负载。第四章将阐述如何通过Warp级并行将12次FK调用的等效串行开销压缩至约2次FK的水平。

位姿误差与收敛判据

目标位姿定义

批量求解的输入为 N 个目标末端位姿，每个目标以齐次变换矩阵形式给定：

$$\mathbf{T}^d = \begin{bmatrix} \mathbf{R}^d & \mathbf{p}^d \\ \mathbf{0}^\top & 1 \end{bmatrix} \in SE(3)$$

其中 $\mathbf{R}^d \in SO(3)$ 为目标旋转矩阵， $\mathbf{p}^d \in \mathbb{R}^3$ 为目标位置向量。本文的数据资产并非任意随机姿态，而是由同一UR10模型先采样关节角、再通过上式正向生成的可达目标位姿。该生成方式确保每个目标位姿在运动学上至少存在一组精确关节解（即采样时使用的关节角），使收敛率可作为求解器有效性的无偏度量——任何未收敛情形均可归因于求解器算法本身的局限性，而非目标位姿本身不可达。

位置误差与姿态误差

给定当前关节角 $\mathbf{q}$ 下的末端位姿 $\mathbf{T}_{EE}(\mathbf{q})$ ，其与目标位姿 $\mathbf{T}^d$ 之间的6维误差向量 $\mathbf{e}(\mathbf{q}) \in \mathbb{R}^6$ 由位置残差与姿态残差拼接而成。

位置误差定义为欧氏空间中的简单向量差：

$$\mathbf{e}_p(\mathbf{q}) = \mathbf{p}^d - \mathbf{p}(\mathbf{q}) \in \mathbb{R}^3$$

姿态误差需在 $SO(3)$ 的流形结构上度量。定义相对旋转矩阵 $\mathbf{R}_{err} = \mathbf{R}^d \mathbf{R}(\mathbf{q})^\top$ （表示从当前姿态到目标姿态的旋转），其对应的李代数向量由对数映射提取：

$$\mathbf{e}_R(\mathbf{q}) = \text{Log}(\mathbf{R}^d \mathbf{R}(\mathbf{q})^\top)^\vee \in \mathbb{R}^3$$

其中 $\text{Log} : SO(3) \rightarrow \mathfrak{so}(3)$ 为李群到李代数的对数映射， $(\cdot)^\vee : \mathfrak{so}(3) \rightarrow \mathbb{R}^3$ 为反对称矩阵到向量的逆映射。上式的几何含义为：将当前姿态旋转至目标姿态所需绕单一轴旋转的角度向量，其方向为等效旋转轴、模长为旋转角 θ_R 。与直接使用欧拉角差或四元数差相比，该定义具有**流形结构保距性**—— $\|\mathbf{e}_R\|_2$ 等于 $SO(3)$ 上的测地线距离，避免了欧拉角表征的万向节奇异性和四元数双覆盖歧义。

旋转角 θ_R 可由相对旋转矩阵的迹显式计算：

$$\theta_R = \cos^{-1} \left(\frac{\text{tr}(\mathbf{R}^d \mathbf{R}(\mathbf{q})^\top) - 1}{2} \right), \quad 0 \leq \theta_R \leq \pi$$

总6维位姿误差将位置残差与姿态残差拼接，并通过对角权重矩阵 $\mathbf{W}$ 平衡量纲差异：

$$\mathbf{e}(\mathbf{q}) = [\mathbf{e}_p(\mathbf{q}); \mathbf{e}_R(\mathbf{q})] \in \mathbb{R}^6, \quad \mathbf{W} = \text{diag}(w_p, w_p, w_p, w_R, w_R, w_R)$$

其中 w_p 为位置权重， w_R 为姿态权重。默认配置取 $w_p = w_R = 1.0$ 。

收敛判据

迭代过程在同时满足位置与姿态阈值时判定为收敛：

$$\|\mathbf{e}_p(\mathbf{q})\|_2 < \varepsilon_p, \quad \theta_R < \varepsilon_R$$

为评估不同任务精度要求下的求解器性能边界，本文设置 Loose、Medium 和 Strict 三档收敛阈值：

等级	ε_p (位置)	ε_R (姿态)	定位
Loose	0.030 m (30 mm)	0.1745 rad (10°)	宽松任务 / 候选生成
Medium	0.010 m (10 mm)	0.0873 rad (5°)	主 benchmark 默认口径
Strict	0.005 m (5 mm)	0.0175 rad (1°)	严格阈值压力验证

其中 Medium 作为论文主 benchmark 的默认收敛判据。位置与姿态误差具有不同物理量纲，本文统一采用独立阈值进行收敛判定，避免单一加权误差掩盖某一项未收敛的情况。

阻尼最小二乘IK迭代模型

非线性最小二乘问题建模

对于单个目标位姿 $\mathbf{T}^d$ ，其IK解等价于以下非线性加权最小二乘问题的最优解：

$$\min_{\mathbf{q}} \frac{1}{2} \|\mathbf{W} \mathbf{e}(\mathbf{q})\|_2^2 = \frac{1}{2} \mathbf{e}(\mathbf{q})^\top \mathbf{W}^2 \mathbf{e}(\mathbf{q})$$

其中 $\mathbf{e}(\mathbf{q})$ 由前述公式定义。该问题为6维流形上的无约束（除关节限位外）非线性优化——目标函数高度非凸，存在多个局部极小值对应不同的构型分支（如上肘/下肘、翻腕/不翻腕），解的质量依赖于初始种子 $\mathbf{q}^{(0)}$ 的选取。

Gauss-Newton线性化与DLS正则化

在当前迭代点 $\mathbf{q}^{(k)}$ 处对 $\mathbf{e}(\mathbf{q})$ 进行一阶泰勒展开：

$$\mathbf{e}(\mathbf{q}^{(k)} + \Delta \mathbf{q}) \approx \mathbf{e}(\mathbf{q}^{(k)}) + \mathbf{J}(\mathbf{q}^{(k)}) \Delta \mathbf{q}$$

其中 $\mathbf{J}(\mathbf{q}) \in \mathbb{R}^{6 \times 6}$ 为位姿误差对关节角的雅可比矩阵 ($J_{ij} = \partial e_i / \partial q_j$)。代入上式并对 $\Delta \mathbf{q}$ 求梯度，令梯度为零得到 Gauss-Newton 正规方程：

$$\mathbf{J}^\top \mathbf{W}^2 \mathbf{J} \Delta \mathbf{q} = \mathbf{J}^\top \mathbf{W}^2 \mathbf{e}(\mathbf{q}^{(k)})$$

当 $\mathbf{J}$ 接近奇异时（如机械臂处于或接近奇异构型、接近关节限位时），矩阵 $\mathbf{J}^\top \mathbf{W}^2 \mathbf{J}$ 的条件数急剧增大，上式产生的步长 $\Delta \mathbf{q}$ 可能趋于无界。阻尼最小二乘法（Damped Least-Squares, DLS）引入二次阻尼项对该病态性进行正则化：

$$\Delta \mathbf{q} = \arg \min_{\Delta \mathbf{q}} \|\mathbf{W}(\mathbf{J} \Delta \mathbf{q} - \mathbf{e})\|_2^2 + \lambda^2 \|\Delta \mathbf{q}\|_2^2$$

其中 $\lambda > 0$ 为阻尼系数。阻尼项 $\lambda^2 \|\Delta \mathbf{q}\|_2^2$ 同时起到两个作用：（1）压制大步长，防止线性近似失效区域内的振荡；（2）在 $\mathbf{J}$ 的零空间方向上提供唯一解（最小范数解）。令上式对 $\Delta \mathbf{q}$ 的导数为零，得到 DLS 正规方程：

$$(\mathbf{J}^\top \mathbf{W}^2 \mathbf{J} + \lambda^2 \mathbf{I}) \Delta \mathbf{q} = \mathbf{J}^\top \mathbf{W}^2 \mathbf{e}(\mathbf{q}^{(k)})$$

阻尼正规矩阵与梯度向量的定义

记 $\mathbf{H}$ 为阻尼正规矩阵（Gauss-Newton 近似海森）、 $\mathbf{g}$ 为梯度向量：

$$\mathbf{H} = \mathbf{J}^\top \mathbf{W}^2 \mathbf{J} + \lambda^2 \mathbf{I} \in \mathbb{R}^{6 \times 6}, \quad \mathbf{g} = \mathbf{J}^\top \mathbf{W}^2 \mathbf{e}(\mathbf{q}) \in \mathbb{R}^6$$

由于 $\mathbf{W}$ 为对角正定矩阵且 $\lambda > 0$ ， $\mathbf{H}$ 为对称正定（SPD）矩阵——该性质确保 LDL^T 分解无需选主元即可稳定执行，且 $\Delta \mathbf{q}$ 为严格下降方向。上式简化为 6×6 线性系统：

$$\mathbf{H} \Delta \mathbf{q} = \mathbf{g}$$

关节更新与限位投影

得到步长 $\Delta \mathbf{q}$ 后，关节角按下式更新并投影至合法关节空间：

$$\mathbf{q}^{(k+1)} = \Pi_Q(\mathbf{q}^{(k)} + \alpha \Delta \mathbf{q})$$

其中 $\Pi_Q(\cdot)$ 为逐维度的关节限位投影算子：

$$[\Pi_Q(\mathbf{q})]_i = \text{clamp}(q_i, q_{i,\min}, q_{i,\max}), \quad i = 1, \dots, 6$$

步长 $\alpha \in (0, 1]$ 由 $\|\Delta \mathbf{q}\|_\infty \leq 0.25$ rad的硬约束确定——若 $\|\Delta \mathbf{q}\|_\infty \leq 0.25$ ，则 $\alpha = 1.0$ （全步长）；否则 $\alpha = 0.25 / \|\Delta \mathbf{q}\|_\infty$ （等比缩放）。该约束的物理依据为：单步关节角变化超过0.25 rad（约14.3°）时，线性近似假设的截断误差可能主导求解方向，导致迭代发散。

数值雅可比矩阵

雅可比矩阵 $\mathbf{J} \in \mathbb{R}^{6 \times 6}$ 的第 j 列（ $j = 1, \dots, 6$ ）通过中心差分格式数值计算：

$$\mathbf{J}_{:,j}(\mathbf{q}) = \frac{\mathbf{e}(\mathbf{q} + \varepsilon \mathbf{e}_j) - \mathbf{e}(\mathbf{q} - \varepsilon \mathbf{e}_j)}{2\varepsilon}$$

其中 $\varepsilon = 10^{-6}$ 为扰动步长， $\mathbf{e}_j$ 为第 j 个标准基向量（仅第 j 维为1，其余为0）。每列需2次独立的FK+位姿误差计算（正向扰动与负向扰动），6列合计12次FK调用——按上式估算，总计算量约 $12 \times 538 \approx 6,456$ 次标量运算。

选择数值雅可比而非解析雅可比的技术依据有三。其一，**通用性**：解析雅可比需为每台机器人的每个关节推导特定的几何雅可比公式（涉及关节轴叉积与坐标系原点差向量），而数值雅可比仅依赖FK函数——适用于任意运动学链，在标准化迁移中无需为模型变更重新推导。其二，**GPU并行补偿**：12次FK的串行开销在6路Warp级并行下等效于约2次FK的延迟（6列由6个lane同时计算），且FK中的三角函数经SFU流水线与其他计算实现有效重叠。其三，**精度充分性**：中心差分格式的截断误差为 $O(\varepsilon^2) \approx 10^{-12}$ 量级（FP64下），远低于迭代收敛容差 10^{-2} 量级，不会引入影响收敛判断的数值噪声。

LM混合阻尼策略概述

阻尼系数 λ 的调度质量直接影响收敛速度与收敛率。本文采用LM（Levenberg-Marquardt）混合阻尼策略，分两阶段差异化控制：**迭代0——距离初始化**，利用初始TCP位置误差 $e_{\text{pos}} = \|\mathbf{e}_p(\mathbf{q}^{(0)})\|$ 计算初始阻尼——远距离目标（ $e_{\text{pos}} > 0.1$ m）采用高阻尼以压制大梯度振荡，近距离目标（ $e_{\text{pos}} \leq 0.1$ m）采用低阻尼以保持二次收敛速度；**迭代1+——Marquardt误差驱动更新**，基于相邻迭代的位置误差比值 $r = e_{\text{pos}}^{(k)} / e_{\text{pos}}^{(k-1)}$ 自适应调整——误差显著减小（ $r < 0.9$ ）时衰减阻尼（ $\times 0.7$ ），误差增大（ $r > 1.1$ ）时增加阻尼（ $\times 2.0$ ），滞回窗口内保持阻尼不变。阻尼全局钳位至 $[10^{-4}, 0.5]$ ，并辅以停滞超驰机制（连续12次无改善时强制 $\lambda \times 5.0$ ，将搜索方向拉回梯度下降域）。该策略的完整参数选取依据与消融验证见第四章。

LDL^T分解与小矩阵系统

矩阵规模与对称正定性

上式定义了一个**固定规模**的线性系统： $\mathbf{H} \in \mathbb{R}^{6 \times 6}$ ， $\mathbf{g} \in \mathbb{R}^6$ ， $\Delta \mathbf{q} \in \mathbb{R}^6$ 。矩阵规模不随机械臂自由度数变化（对于6-DOF串联臂恒为 6×6 ），也不随批量目标数 N 增长——每个目标的DLS迭代始终求解同一规模的线性系统。

$\mathbf{H}$ 的对称正定性由两项保证：（1） $\mathbf{J}^\top \mathbf{W}^2 \mathbf{J}$ 为半正定（ $\mathbf{W}^2$ 为对角正定，二次型 $\mathbf{v}^\top \mathbf{J}^\top \mathbf{W}^2 \mathbf{J} \mathbf{v} = \|\mathbf{W} \mathbf{J} \mathbf{v}\|_2^2 \geq 0$ ）；（2）阻尼项 $\lambda^2 \mathbf{I}$ （ $\lambda > 0$ ）使全体特征值严格正偏移 λ^2 ，确保 $\mathbf{H} \succ 0$ 。对称正定性使LDL^T分解（一种无平方根、无需选主元的Cholesky变体）成为该系统的自然求解方法。

LDL^T分解与求解步骤

LDL^T分解将H分解为单位下三角矩阵L、对角矩阵D及L^T的乘积：

$$\mathbf{H} = \mathbf{L}\mathbf{D}\mathbf{L}^\top$$

其中 $\mathbf{L} = [\ell_{ij}] \in \mathbb{R}^{6 \times 6}$ ， $\ell_{ii} = 1$ （单位对角线）， $\ell_{ij} = 0$ 对 $i < j$ （下三角）； $\mathbf{D} = \text{diag}(d_1, \dots, d_6)$ ， $d_i > 0$ 由 $\mathbf{H} \succ 0$ 保证。分解算法为三重循环的逐列约化——外层 $j = 0, \dots, 5$ 遍历列，内层 $k = 0, \dots, j - 1$ 累加已分解列的贡献，最内层 $i = j + 1, \dots, 5$ 更新非对角元并除以 d_j 。由于矩阵规模固定为6，三重循环的边界均为编译时常量，可由NVCC编译器完全展开（\#pragma unroll），消除循环控制指令开销。

获得L与D后，线性系统HΔq = g分三步求解：

步骤	公式	说明
前代	$\mathbf{L}\mathbf{y} = \mathbf{g}$	逐行自上而下，每行约 <i>i</i> 次乘减
对角缩放	$\mathbf{D}\mathbf{z} = \mathbf{y}$	$z_i = y_i/d_i$ ，共6次除法
回代	$\mathbf{L}^\top\Delta\mathbf{q} = \mathbf{z}$	逐行自下而上，每行约6 - <i>i</i> - 1次乘减

分解阶段的计算量为对角元更新 $\sum_{j=0}^5 j = 15$ 次乘加、非对角元更新 $\sum_{j=0}^5 \sum_{i=j+1}^5 j = 20$ 次乘加、以及L缩放 $\sum_{j=0}^5 (6 - j - 1) = 15$ 次除法。求解阶段的前代与回代各15次乘减，对角缩放6次除法。总计86次标量运算，其中65次可映射为GPU FMA（fused multiply-add）指令——单周期完成的 $a \times b + c$ 融合操作——21次为除法（非FMA-able）。在1.5–2.0 GHz SM频率下，86次标量运算在寄存器中的执行时间约为0.1 μs。

固定小规模系统对GPU实现的结构性优势

由于UR10为6自由度串联机械臂，单个IK查询在每次DLS迭代中只需求解固定规模的(6x6)对称正定线性系统。该规模不足以有效利用通用矩阵库，但适合在单个CUDA block内进行寄存器级展开计算。

具体而言，6 × 6固定规模带来四重结构性优势。****第一，消除库调用开销。***对6 × 6极小矩阵而言，通用线性代数库的kernel launch与调度开销可能超过实际分解计算量——cuBLAS等库的最小优化规模远大于6 × 6（其Warp级分块策略针对 $n \geq 32$ 的矩阵优化）。寄存器手动实现完全消除这一开销。**第二，全寄存器驻留。**L矩阵（36个double）、D对角元（6个double）、中间向量y, z, Δq（18个double）共计60个double——约占120个32位寄存器，在Ada Lovelace 255寄存器/线程上限内，无局部内存溢出（zero spilling）。****第三，编译时完全展开。***矩阵维度6为编译时常量，三重循环的迭代次数（ $j = 0..5, k = 0..j - 1, i = j + 1..5$ ）完全确定，NVCC \#pragma unroll将全部循环展开为直线代码序列，消除分支预测失败与循环控制开销。**第四，LDL^T选型优于Cholesky。**H = LL^T的Cholesky分解需6次平方根运算——GPU上平方根指令的吞吐量

约为FMA的1/4至1/8——而LDL^T以额外的 $O(n^2)$ 次乘加运算替代平方根，在 $n = 6$ 的规模下新增运算量极小（约15次FMA），但避免了6次低吞吐的平方根指令，整体延迟更低。

上述优势的共同前提是矩阵规模固定且足够小——若扩展至7-DOF冗余机械臂（ $\mathbf{H} \in \mathbb{R}^{7 \times 7}$ ），寄存器占用量将增至约160个寄存器，仍处于可行区间（ < 255 ）；但若扩展至高冗余度（ $n \geq 12$ ），寄存器压力将迫使部分变量溢出至共享内存，性能特性需重新评估。本文第7章将讨论7DOF Panda 的迁移方法与正确性验证。

批量IK任务定义与评价指标

批量任务的形式化定义

给定 N 个目标末端位姿集合 $\{\mathbf{T}_i^d\}_{i=1}^N$ 与对应的 N 个初始关节角种子 $\{\mathbf{q}_i^{(0)}\}_{i=1}^N$ ，批量逆运动学求解定义为：为每个 $i \in \{1, \dots, N\}$ 寻找关节角 $\mathbf{q}_i^* \in \mathbb{R}^6$ ，使其满足上式的收敛判据，或经 $K_{\max}$ 次DLS迭代（ $K_{\max} = 50$ ）后终止并返回历史最优解（以最终位姿误差 $\|\mathbf{e}(\mathbf{q}^{(K)})\|_2$ 最小者）。

形式化表述为：

$$\begin{aligned} \text{求解: } \mathbf{q}_i^* &= \arg \min_{\mathbf{q} \in \mathcal{Q}} \|\mathbf{W} \mathbf{e}_i(\mathbf{q}; \mathbf{T}_i^d)\|_2^2, \quad i = 1, \dots, N \\ \text{约束: } \mathbf{q}^{(k+1)} &= \Pi_{\mathcal{Q}}(\mathbf{q}^{(k)} + \alpha \Delta \mathbf{q}^{(k)}), \quad k = 0, \dots, K_{\max} - 1 \\ \Delta \mathbf{q}^{(k)} &= (\mathbf{J}^\top \mathbf{W}^2 \mathbf{J} + \lambda^2 \mathbf{I})^{-1} \mathbf{J}^\top \mathbf{W}^2 \mathbf{e}(\mathbf{q}^{(k)}) \end{aligned}$$

该问题的关键结构性质为**跨目标完全解耦**：目标 i 的求解过程不依赖目标 j （ $j \neq i$ ）的任何中间结果——无数据共享，无同步需求，无迭代间通信。这一独立性是CUDA block级并行映射的数学基础： N 个目标可分配至 N 个独立的CUDA block，各block完全并行执行，无block间同步或通信。

性能评价指标

批量IK求解器的性能由以下三项指标综合衡量：

****（1）吞吐量（Throughput）。****单位时间内处理的目标总数（Query Throughput），定义为：

$$\text{Throughput} = \frac{N}{T_{\text{total}}} \quad (\text{targets/s, t/s})$$

其中 N 为目标总数， T_{total} 为总耗时。收敛率（ConvRate）单独报告。对于收敛率不为1的配置，成功求解吞吐（Successful Throughput）可由 $\text{Throughput} \times \text{ConvRate}$ 得到。

****（2）收敛率（Convergence Rate）。****在给定初始种子 $\mathbf{q}_i^{(0)}$ 下满足收敛判据的目标比例：

$$\text{CR} = \frac{N_{\text{converged}}}{N} \times 100\%$$

收敛率反映求解器在当前种子策略下的鲁棒性——高收敛率意味着对关节空间各区域的覆盖充分，低收敛率则表明种子策略存在覆盖盲区或DLS迭代易陷入局部极小。需注意收敛率与种子策略紧密耦合：同一求解器在不同种子策略下的收敛率可有显著差异，因此收敛率的比较必须在统一种子条件下进行。

****（3）平均迭代次数（Average Iterations）。****收敛目标从初始种子到达收敛判据所需的平均DLS迭代次数：

$$\bar{K}_{\text{conv}} = \frac{1}{N_{\text{converged}}} \sum_{i:\text{converged}} K_i$$

平均迭代次数反映求解器的收敛效率——在同等收敛率下，更低的平均迭代次数意味着更高效的阻尼调度与更优的搜索方向，直接降低每目标的平均计算开销（每轮迭代需12次FK调用 + 1次 6×6 LDL^T求解 + 1次阻尼正规矩阵/梯度构造）。

时间测量口径

批量GPU求解器的性能数字高度依赖于时间边界的定义。为消除跨求解器比较中的口径歧义，本文明确定义三种时间测量范围：

****kernel_time_only**（纯内核时间）：******仅统计GPU kernel从发射到完成的设备端执行时间，由CUDA event的 `cudaEventElapsedTime`在`ik\batch_solve` kernel的首尾事件间测量。不含主机端预处理、不含 H2D/D2H数据传输、不含设备同步延迟。该口径反映kernel本身的并行效率，是GPU算法优化的核心关注对象。

****gpu_end_to_end_time**（GPU端到端时间）：******在`kernel_time_only`基础上增加主机至设备的数据传输时间（目标位姿、种子关节角的H2D传输）与设备至主机的回读时间（收敛关节角、误差统计的D2H传输），以及 `cudaDeviceSynchronize` 的同步等待时间。该口径反映GPU求解器的实用可用性能——包含完成一次完整批量求解所必需的全部GPU侧耗时。

****host_api_total_time**（主机API总时间）：******在`gpu_end_to_end_time`基础上增加主机端的预处理耗时，包括种子数组的CPU侧组装、设备内存的分配与释放、结果向量的CPU侧验证（关节限位检查、FK复算验证）。该口径反映从用户调用API到获得可用结果的完整端到端延迟——是最接近应用层感知的时间口径。

三种口径的分层定义用于揭示性能瓶颈的归属：若`kernel_time_only`与`gpu_end_to_end_time`之间存在显著差距，表明数据传输或设备同步为瓶颈；若`gpu_end_to_end_time`与`host_api_total_time`之间存在显著差距，表明主机端预处理为瓶颈。该分层分析方法用于区分 kernel 计算、数据传输和主机 API 开销对整体性能的影响。

CUDA并行映射框架预览

本章建立的数学模型到GPU并行执行的映射遵循以下基本原则，其详细工程实现见第四章。

令CUDA的线程层次结构为：Grid维度 $(N, 1, 1)$ ，Block维度 $(128, 1, 1)$ ，每个Block包含4个Warp（每Warp 32线程）。映射关系由以下索引公式定义：

$$b = \text{blockIdx.x} \in \{0, \dots, N - 1\}, \quad t = \text{threadIdx.x} \in \{0, \dots, 127\}$$
$$w = \lfloor t/32 \rfloor \in \{0, 1, 2, 3\}, \quad \ell = t \bmod 32 \in \{0, \dots, 31\}$$

各Warp的职责分配直接对应本章各项数学公式的计算任务：

表1 Warp职责分配与数学公式对应关系

Warp	Lane范围	对应公式	计算任务	并行度
W0	0--31	前述FK、位姿误差、LDL ^T 求解、LM阻尼更新、收敛判定	FK链式乘法、位姿误差计算、LDL ^T 求解、LM阻尼更新、收敛判定	Lane 0串行控制
W1	32--63	上式	数值雅可比6列并行组装（每列1 lane）	6路数据并行

Warp	Lane范围	对应公式	计算任务	并行度
W2	64--95	上式	阻尼正规矩阵 $\mathbf{H}$ 与梯度向量 $\mathbf{g}$ 构造 (每元素1 lane)	36路数据并行
W3	96- -127	前述关节更新、限位投影、步长裁剪	关节角步长更新、限位投影、步长裁剪	6路数据并行

该映射方案的核心优势在于： N 个目标的全部DLS迭代在**单次kernel launch**中完成——上式的 N 个独立优化问题由 N 个block并行求解，每个block内部的4个Warp协作完成上式的构造与求解。与逐目标多次kernel launch的方案相比，该设计消除了kernel launch累积开销——该开销恰是现有GPU求解器（如cuRobo）在中小批量场景下的主要性能瓶颈。第四章将依次阐述该映射框架中每个Warp的CUDA工程实现、共享内存布局的Bank冲突消除设计、以及寄存器级LDL^T的完整伪代码。

4 CUDA 并行求解器设计

第三章已将单次IK迭代归结为求解固定规模的 6×6 对称正定线性系统 $\mathbf{H}\Delta\mathbf{q} = \mathbf{g}$ ，并建立了从数学公式到GPU线程的映射框架。本章在此框架基础上，从Block级并行映射、寄存器级LDL^T求解、共享内存布局、混合精度计算路径和自适应阻尼策略五个维度，系统阐述CUDA并行求解器的工程设计与硬件适配推导。每一项设计决策均以GPU硬件约束（寄存器文件容量、Bank数量、SM共享内存上限）为边界条件，以Nsight Compute实测数据为验证依据。

批量目标的Block级并行映射

Grid到Block的一维索引映射

批量IK的核心特征是 N 个目标位姿的求解完全解耦——每个目标的DLS迭代过程不依赖其他目标的中间结果。这一天然并行性可直接映射到CUDA的Block级并行模型。设批量规模为 N ，目标位姿矩阵为 $\mathbf{T}_{\text{tgt}}^{(b)} \in SE(3)$ ($b = 0, 1, \dots, N - 1$)，关节角种子为 $\mathbf{q}^{(b,0)} \in \mathbb{R}^6$ ，则Grid维度定义为：

$$\text{Grid} = (N, 1, 1), \quad \text{Block} = (128, 1, 1)$$

其中Block索引 $b = \text{blockIdx.x}$ 直接映射到第 b 个批量目标，Block内128个线程协作完成该目标的全部DLS迭代。Grid中 N 个Block完全并行发射，零Block间通信，零全局同步——每个Block仅在其结束时通过全局内存写回收敛关节角 $\mathbf{q}^{(b,*)}$ 、误差向量 $\mathbf{e}^{(b,*)}$ 和迭代次数 $k^{(b)}$ 。

128线程4-Warp分工的约束驱动选择

Block内部线程到计算任务的映射需同时满足三项硬件约束：寄存器文件容量、共享内存容量和Warp调度效率（32线程/Warp的SIMT执行模型）。本节通过约束驱动的排除法论证128线程4-Warp方案的必要性。具体寄存器分配与occupancy以Nsight Compute报告为准。

****候选方案A：每线程一任务（Fine-grained）。****每个线程独立完成一个目标的全部DLS迭代。每个线程需独立存储：雅可比矩阵 $\mathbf{J} \in \mathbb{R}^{6 \times 6}$ （36个FP64元素，等价72个32位寄存器）、阻尼正规矩阵 $\mathbf{H} \in \mathbb{R}^{6 \times 6}$ （72个寄存器）、6段FK的中间 4×4 齐次变换矩阵（ $6 \times 16 = 96$ 个FP64元素，等价192个寄存器）、以及误差/梯度/步长向量（18个FP64元素，等价36个寄存器）。寄存器需求估算：

$$R_{\text{per-thread}}^{(\text{A})} = 72 + 72 + 192 + 36 \approx 372 \gg 255$$

远超Ada Lovelace的255寄存器/线程硬件上限。编译器被迫将溢出变量写入local memory（物理上位于DRAM，延迟约400周期），性能将严重下降。

****候选方案B：每Block一任务（Cooperative）。****每个Block（128线程，4 Warp）协作完成一个目标的求解。 6×6 矩阵（**J**和**H**）置于共享内存中供Block内线程协作访问，每线程仅需存储其FK扰动中间结果、误差/步长向量和控制变量：

$$R_{\text{per-thread}}^{(B)} = \underbrace{64}_{\text{FK中间变量}} + \underbrace{12}_{\text{误差/步长}} + \underbrace{8}_{\text{控制/索引}} = 84 \ll 255$$

共享内存占用量同样远低于硬件上限：

$$S_{\text{per-block}} = \underbrace{768}_{\text{J+H矩阵}} + \underbrace{848}_{\text{关节/位姿/缓冲}} = 1,616 \text{ Bytes} \ll 98,304 \text{ Bytes/SM}$$

****三重约束验证。 ****128线程/Block的配置通过以下三项验证：（1）Warp对齐： $128 = 4 \times 32$ ，4个Warp恰好对应4项主要任务类别，无Warp资源浪费；（2）共享内存：1,616 B静态共享内存/Block，远低于SM共享内存容量；（3）寄存器：Nsight Compute实测显示该kernel使用94 registers/thread，无local memory spill。以上配置的achieved occupancy约32%–33%，主要受寄存器压力限制。

Warp级任务分配的形式化定义

Block内线程索引 $t = \text{threadIdx.x} \in [0, 127]$ ，Warp索引与Lane索引定义为：

$$w = \left\lfloor \frac{t}{32} \right\rfloor, \quad l = t \bmod 32$$

四个Warp的任务分配如表所示，该分配直接对应第三章前述公式中各计算步骤的并行性特征。

表2 Warp级任务分配与执行模式

Warp	Lane范围	主要职责	并行粒度	执行模式
$w = 0$	0–31	FK链式乘法、位姿误差计算、LDL ^T 求解、LM阻尼更新、收敛判定	Lane 0串行执行	控制流
$w = 1$	32–63	数值雅可比组装：Lane $32 + j$ 负责第 j 列（ $j = 0, \dots, 5$ ），每列2次FK扰动计算	6路数据并行	数据并行
$w = 2$	64–95	阻尼正规矩阵构造：Lane $64 + (i \times 6 + j)$ 负责 H _{<i>ij</i>} 元素的内积计算，36元素各一线程	36路数据并行	数据并行
$w = 3$	96–127	梯度向量 g 计算、关节更新 $\mathbf{q}^{(k+1)} = \Pi_{\mathcal{Q}}(\mathbf{q}^{(k)} + \alpha \Delta \mathbf{q})$ 、步长裁剪	6路数据并行	数据并行

该映射方案的核心优势在于： N 个目标的全部DLS迭代在**单次kernel launch**中完成——消除了第三章上式所指出的逐目标独立kernel launch的累积启动开销，该开销在cuRobo的基准测试中占端到端延迟的15–30\。

DLS小矩阵系统的寄存器级LDL^T求解

从DLS正规方程到6 × 6线性系统

第三章上式将每次DLS迭代归结为阻尼正规方程：

$$(\mathbf{J}^\top \mathbf{W}^2 \mathbf{J} + \lambda^2 \mathbf{I}) \Delta \mathbf{q} = \mathbf{J}^\top \mathbf{W}^2 \mathbf{e}(\mathbf{q})$$

记 $\mathbf{H} = \mathbf{J}^\top \mathbf{W}^2 \mathbf{J} + \lambda^2 \mathbf{I} \in \mathbb{R}^{6 \times 6}$ 为阻尼正规矩阵， $\mathbf{g} = \mathbf{J}^\top \mathbf{W}^2 \mathbf{e}(\mathbf{q}) \in \mathbb{R}^6$ 为加权梯度向量。由于 $\mathbf{W}$ 为对角正定矩阵且 $\lambda > 0$ ， $\mathbf{H}$ 为对称正定矩阵（SPD），可通过 $\text{LDL}^\top$ 分解（无需选主元）求解：

$$\mathbf{H} \Delta \mathbf{q} = \mathbf{g}$$

$$\mathbf{H} = \mathbf{L} \mathbf{D} \mathbf{L}^\top$$

其中 $\mathbf{L} \in \mathbb{R}^{6 \times 6}$ 为单位下三角矩阵（ $L_{ii} = 1$ ）， $\mathbf{D} = \text{diag}(d_0, \dots, d_5)$ 为对角矩阵。求解分三步：

$$\mathbf{L} \mathbf{y} = \mathbf{g}, \quad \mathbf{D} \mathbf{z} = \mathbf{y}, \quad \mathbf{L}^\top \Delta \mathbf{q} = \mathbf{z}$$

寄存器驻留设计的工程论证

6×6 线性系统的求解位于DLS迭代关键路径的串行瓶颈。在CUDA环境下，存在两条技术路线：(a) 调用外部线性代数库（如cuBLAS的GEMM/TRSM接口）；(b) 寄存器级手动实现。本文基于以下定量分析选择路线 (b)。

对 6×6 极小矩阵而言，通用线性代数库的kernel launch与调度开销可能超过实际分解计算量。cuBLAS的最小优化规模远大于 6×6 （其Warp级分块策略针对 $n \geq 32$ 的矩阵优化）， 6×6 矩阵在cuBLAS中无法利用Tensor Core或高效的Warp级tiling策略。因此，本文采用寄存器级专用实现以避免额外库调用开销。

路线 (b) 将全部矩阵元素 $\mathbf{H}$ 、 $\mathbf{L}$ 、 $\mathbf{D}$ 、中间向量 $\mathbf{y}$ 、 $\mathbf{z}$ 和最终解 $\Delta \mathbf{q}$ 置于寄存器中（总计 $36 + 36 + 6 + 6 + 6 + 6 = 96$ 个FP64元素 = 768 bytes）， $\text{LDL}^\top$ 求解阶段本身不访问全局内存和共享内存。鉴于 $\text{LDL}^\top$ 由Lane 0 串行执行而其余127个线程处于等待状态，该方案不增加Warp的关键路径延迟——Lane 0的寄存器独占使用不会引发与其他Lane的寄存器争用。

$\text{LDL}^\top$ 分解的寄存器级实现与运算计数

$\text{LDL}^\top$ 分解的完整寄存器级实现包含四个阶段。以下以伪代码形式给出各阶段的计算步骤与运算量分析。

阶段一： $\text{LDL}^\top$ 分解（ $\mathbf{H} = \mathbf{L} \mathbf{D} \mathbf{L}^\top$ ）。 算法从PaddedMat6x8共享内存布局（见上文）加载 $\mathbf{H}$ 至寄存器数组 $\mathbf{L}$ （以 $\mathbf{L}$ 的下三角部分存储 $\mathbf{L}$ 的严格下三角，对角元在分解过程中转化为 $\mathbf{D}$ 的元素）。对 $j = 0, 1, \dots, 5$ ：

对角元更新：

$$D_j = H_{jj} - \sum_{k=0}^{j-1} L_{jk}^2 \cdot D_k$$

非对角元更新（ $i = j + 1, \dots, 5$ ）：

$$L_{ij} = \frac{1}{D_j} \left(H_{ij} - \sum_{k=0}^{j-1} L_{ik} \cdot L_{jk} \cdot D_k \right)$$

阶段二：前代（ $\mathbf{L} \mathbf{y} = \mathbf{g}$ ）。 对 $i = 0, 1, \dots, 5$ ：

$$y_i = g_i - \sum_{j=0}^{i-1} L_{ij} \cdot y_j$$

阶段三：对角缩放 ($\mathbf{D}\mathbf{z} = \mathbf{y}$)。对 $i = 0, 1, \dots, 5$:

$$z_i = y_i / D_i$$

阶段四：回代 ($\mathbf{L}^\top \Delta \mathbf{q} = \mathbf{z}$)。对 $i = 5, 4, \dots, 0$:

$$\Delta q_i = z_i - \sum_{j=i+1}^5 L_{ji} \cdot \Delta q_j$$

以上四阶段运算量的严格逐项统计见表。

表3 LDL^T求解器各阶段运算量统计

阶段	循环范围	标量运算次数	可 FMA 映射
对角元更新	$j = 0..5, k = 0..j - 1$	$\sum_{j=0}^5 j = 15$	15
非对角元更新	$j = 0..5, i = j + 1..5, k = 0..j - 1$	$\sum_{j=0}^5 \sum_{i=j+1}^5 j = 20$	20
L_{ij} 缩放 (除 D_j)	$j = 0..5, i = j + 1..5$	$\sum_{j=0}^5 (6 - j - 1) = 15$	0 (除法)
前代求解	$i = 0..5, j = 0..i - 1$	$\sum_{i=0}^5 i = 15$	15
对角缩放	$i = 0..5$	6	0 (除法)
回代求解	$i = 5..0, j = i + 1..5$	$\sum_{i=0}^5 (6 - i - 1) = 15$	15
合计		86	65

总计86次标量运算中，65次为乘加/乘减运算，可映射为单周期FMA指令 (fused multiply-add, $a \times b + c$)；21次为除法，由SFU流水线执行。LDL^T求解阶段本身不访问全局内存和共享内存——全部运算在寄存器内完成。在1.5--2.0 GHz SM频率下，86次标量运算的完成时间约0.05--0.10 μs ，与Nsight Compute实测的LDL^T段延迟 ($< 0.15 \mu\text{s}$) 一致。

$$\text{FP64 FLOP}_{\text{LDL}^T} = 65 \times 2 + 21 \approx 151 \text{ (含除法)} \approx 130 \text{ (仅FMA)}$$

作为对照，通用线性代数库求解极小矩阵时，kernel launch与全局内存往返等开销可能远超实际分解计算量；寄存器LDL^T求解阶段本身不产生全局内存流量——对DRAM带宽受限的批量IK场景具有关键意义。

共享内存布局与PaddedMat6x8设计

Bank冲突的数学模型

GPU共享内存由32个Bank组成（Bank 0–31），每个Bank位宽4 bytes。FP64（double，8 bytes）元素跨越2个连续Bank。当同一Warp内的多个线程访问同一Bank的不同地址时，发生Bank冲突—— k 路冲突意味着访问被串行化为 k 次，有效带宽降至 $1/k$ 。

对于以自然行优先布局存储在共享内存中的 6×6 Jacobian矩阵 $\mathbf{J} = [j_{r,c}]$ ($r, c \in \{0, 1, \dots, 5\}$)，行步长为6个FP64元素 = $6 \times 8 = 48 \text{ bytes} = 12 \text{ 个Bank}$ 。第 r 行第 c 列元素的Bank索引为：

$$B(r, c) = (r \times 12 + c \times 2) \bmod 32$$

核心问题出现在按列访问时——在阻尼正规矩阵构造阶段（Warp 2负责），线程需计算 $\mathbf{J}$ 的第 i 行与第 j 行的加权内积 $\mathbf{H}_{ij} = \sum_{k=0}^5 w_k^2 J_{ki} J_{kj}$ ，涉及对 $\mathbf{J}$ 列的并发读取。以列 c 为例，6个元素访问的Bank序列为 $\{2c, 12 + 2c, 24 + 2c, 36 + 2c \equiv 4 + 2c, 48 + 2c \equiv 16 + 2c, 60 + 2c \equiv 28 + 2c\}$ （模32）。由于 $\gcd(12, 32) = 4$ ，Bank访问模式以 $32/4 = 8$ 为周期重复——在6行访问中，必然发生Bank索引的模冲突，导致2–3路Bank冲突。Nsight Compute实测确认该冲突模式：共享内存吞吐量为理论峰值的约1/3。

PaddedMat6x8的Bank冲突消除

PaddedMat6x8将矩阵行步长从6扩展至8（元素），即8个FP64元素 $\times 8 \text{ bytes} = 64 \text{ bytes} = 16 \text{ 个Bank}$ 。形式化定义：对于原始矩阵 $\mathbf{J} \in \mathbb{R}^{6 \times 6}$ ，其PaddedMat6x8表示为 $\tilde{\mathbf{J}} \in \mathbb{R}^{6 \times 8}$ ：

$$\tilde{J}_{r,c} = \begin{cases} J_{r,c}, & 0 \leq c < 6 \\ 0, & c = 6, 7 \end{cases}, \quad r \in \{0, 1, \dots, 5\}$$

共享内存中的地址映射为：

$$\text{addr}(r, c) = r \times 8 + c, \quad r \in [0, 5], c \in [0, 7]$$

在新布局下，第 r 行第 c 列的Bank索引为：

$$B'(r, c) = (r \times 16 + c \times 2) \bmod 32$$

由于 $\gcd(16, 32) = 16$ ，Bank访问模式以 $32/16 = 2$ 为周期。关键性质：**偶数行**（ $r = 0, 2, 4$ ）的起始Bank索引为 $c \times 2$ （Bank 0–15范围），**奇数行**（ $r = 1, 3, 5$ ）的起始Bank索引为 $16 + c \times 2$ （Bank 16–31范围）。两组使用的Bank集合**完全不重叠**。在LDL^T分解和阻尼正规矩阵构造等核心计算中——这些操作每次仅涉及 ≤ 2 行的同时访问——因此可消除stride=6布局导致的主要系统性Bank冲突。该结论已由Nsight Compute实测确认：PaddedMat6x8布局下共享内存Bank冲突显著减少，但完整kernel中仍存在其他共享内存访问路径，NCU仍记录到少量残余bank conflict。

存储代价的定量评估

PaddedMat6x8的额外存储代价： $6 \times (8 - 6) \times 8 = 96 \text{ bytes/矩阵}$ 。 $\mathbf{J}$ 和 $\mathbf{H}$ 两个矩阵合计增加 $96 \times 2 = 192 \text{ bytes}$ 共享内存占用。在每SM 100 KB（102,400 bytes）共享内存容量中，该额外开销占比为：

$$\frac{192}{102,400} \times 100\% \approx 0.19\% < 0.2\%$$

以不到0.2%的共享内存容量换取降低主导性的 Bank 冲突——在共享内存带宽作为阻尼正规矩阵构造阶段主要瓶颈的背景下，该代价可忽略不计。PaddedMat6x8的C++封装仅26行代码（无模板依赖），NVCC -O2（sm_89）编译后经Nsight Compute验证：其PTX指令序列与裸指针访问一致——无额外抽象性能开销。

共享内存完整布局

每Block的共享内存分配方案（Nsight Compute实测约1,616 bytes）见表。所有数组均按8元素stride对齐，以确保PaddedMat6x8的Bank冲突消除特性在整个数据路径中一致维持。

表4 每Block共享内存布局

类别	变量	字节数	对齐方式
矩阵（PaddedMat6x8）	$\widetilde{\mathbf{J}}[48]$	384	stride=8
	$\widetilde{\mathbf{H}}[48]$	384	stride=8
关节/位姿缓冲	$\mathbf{q}[8] + \mathbf{T}[16] + \mathbf{T}_{\text{tgt}}[16]$	320	stride=8
误差/梯度/步长	$\mathbf{e}[6] + \mathbf{g}[6] + \Delta\mathbf{q}[6] + \mathbf{q}_{\text{ref}}[6] + \mathbf{q}_{\text{best}}[6]$	240	stride=8
工具变换	$\mathbf{T}_{\text{tcp}}[16] + \mathbf{T}_{\text{tcp,tgt}}[16]$	256	4 × 4矩阵
标量控制变量	converged + iter_count + λ + best_pos_err + stagnation	28	基本类型

混合精度计算路径

FP32/FP64混合策略的设计动机

第三章前述公式定义的完整DLS迭代管线包含两类不同精度敏感度的运算：（a）FK链式乘法、雅可比矩阵构造和阻尼正规矩阵组装——这些运算涉及大量三角函数和矩阵乘法，对FP64精度的需求相对较低；（b）LDL^T线性系统求解——涉及条件数敏感的除法运算和累积求和，对数值精度要求较高。基于这一区分，本文设计了一条FP32/FP64混合精度计算路径。

数据流与精度边界

混合精度数据流的形式化描述如下：

****FP32计算段（低精度、高吞吐）：****前向运动学、位姿误差计算和数值雅可比组装在FP32精度下执行。FP32操作在消费级Ada Lovelace架构上的吞吐显著高于FP64，因此将FK/Jacobian/阻尼正规矩阵主体迁移至FP32可以降低计算成本；LDLT关键路径保留FP64以维持稳定性。FP32数据宽度减半同时降低了共享内存和寄存器空间占用。对于数值雅可比而言， $\epsilon = 10^{-6}$ 的扰动步长在FP32下引入的相对舍入误差约为 10^{-7} 量级，远小于迭代收敛容差（ 10^{-2} 量级），精度损失可忽略。

****FP64累积段（高精度、低吞吐）：****阻尼正规矩阵 $\mathbf{H} = \mathbf{J}^T \mathbf{W}^2 \mathbf{J} + \lambda^2 \mathbf{I}$ 和梯度向量 $\mathbf{g} = \mathbf{J}^T \mathbf{W}^2 \mathbf{e}$ 的构造涉及36个内积的累加——每个内积为6项加权乘积之和。为抑制FP32累加的截断误差传播， $\mathbf{H}$ 和 $\mathbf{g}$ 的元素以FP64

精度累积。

****FP64求解段（高精度、低吞吐）：****LDL^T分解与求解（前述公式）全程使用FP64精度——LDL^T中的除法运算（ $L_{ij} = \dots / D_j$ ）对数值误差敏感：若 D_j 因FP32舍入而偏小，除数误差经后续回代步骤放大，可能导致步长 $\Delta \mathbf{q}$ 的方向偏差。

****FP32更新段（低精度、高吞吐）：****关节角更新 $\mathbf{q}^{(k+1)} = \Pi_{\mathcal{Q}}(\mathbf{q}^{(k)} + \alpha \Delta \mathbf{q})$ 在FP32下完成——步长 $\Delta \mathbf{q}$ 已由FP64 LDL^T高精度确定，FP32关节角更新的舍入误差在单次迭代中约 10^{-7} rad量级，对收敛路径无实质性影响。

完整数据流可概括为：

FP32 FK/Jacobian $\rightarrow$ FP64 H/g累积 $\rightarrow$ FP64 LDL^T $\rightarrow$ FP32 dq更新

精度边界的硬件利用率验证

Nsight Compute对 $N = 100$ 的full profiling数据为混合精度策略提供了硬件级验证。实测FP64流水线利用率为60.7%，FP32流水线利用率为65.7%——表明LDL^T求解段（FP64，仅Lane 0执行）是FP64管道的主要但非唯一使用者，而FP32管道的较高利用率反映了FK和Jacobian组装段在多个Lane上的并行FP32计算。两个精度管道的利用率均未达到饱和（ $< 70\%$ ），表明当前瓶颈不在算术吞吐量，而在指令依赖链和Warp调度延迟——这与LDL^T串行段（Lane 0独占）和共享内存同步点（每次迭代最多7次`__syncthreads()`）的特征一致。

$$\text{FP64利用率} = 60.7\% < 100\%, \quad \text{FP32利用率} = 65.7\% < 100\%$$

自适应阻尼与步长控制策略

阻尼策略的数学形式与GPU实现

第三章前述公式已给出LM混合阻尼策略的完整数学形式。本节从CUDA实现角度论述其在GPU上的工程适配，以及该策略在消融实验中的量化效果。

阻尼参数 λ 的自适应更新全部在Lane 0的串行控制流中完成（Warp 0），仅涉及约15次标量运算（包括一次距离范数计算和若干比较分支），对Block关键路径的延迟贡献可忽略（ $< 0.02 \mu\text{s}/\text{迭代}$ ）。 λ 的当前值、stagnation计数器和历史最优误差存储在共享内存的标量变量区（28 bytes，见表），每次迭代由Lane 0读取、更新并写回，经`__syncthreads()`确保其他Warp在下一次迭代开始时可见新 λ 值。

步长裁剪与关节限位投影

从LDL^T求解得到的步长 $\Delta \mathbf{q}$ 在应用于关节更新前需经两道约束：

****（一）步长幅值裁剪。****当 $\|\Delta \mathbf{q}\|_{\infty} > 0.25 \text{ rad}$ 时，按比例缩放：

$$\Delta \mathbf{q} \leftarrow \Delta \mathbf{q} \cdot \frac{0.25}{\|\Delta \mathbf{q}\|_{\infty}}$$

该阈值（ $0.25 \text{ rad} \approx 14.3^\circ$ ）基于UR10关节的运动范围设定——单次迭代的关节角变化超过此阈值通常表明 $\mathbf{H}$ 条件数恶化或 λ 过低，需通过裁剪防止迭代发散。

****（二）关节限位投影。****将更新后的关节角投影至关节限位可行域 $\mathcal{Q} = [\mathbf{q}_{\min}, \mathbf{q}_{\max}]$ ：

$$\mathbf{q}^{(k+1)} = \Pi_{\mathcal{Q}}(\mathbf{q}^{(k)} + \alpha \cdot \Delta \mathbf{q}), \quad \Pi_{\mathcal{Q}}(q_i) = \text{clamp}(q_i, q_{i,\min}, q_{i,\max})$$

关节限位数据 ($\mathbf{q}_{\min}, \mathbf{q}_{\max} \in \mathbb{R}^6$) 存储于常量内存 ($12 \times 8 = 96$ bytes), 零全局内存访问。

**** (三) 分支对齐 (Branch Alignment).** **当检测到关节角 q_i 在更新后与目标解的候选分支差异超过 π 时, 通过加减 2π 将关节角对齐至最近分支。分支对齐用于处理关节角周期性带来的等价表示问题, 避免数值更新后关节角落入不期望的 2π 分支。分支对齐的判断和修正由Lane 0在关节更新后执行, 涉及6次条件比较和至多6次加减 2π 操作。

消融实验中阻尼与步长控制策略的定位

自适应阻尼、步长钳位和分支对齐的实际影响将在第 6.3 节通过 B0-B6 消融实验量化。简要而言, B3 用于评估自适应阻尼的贡献, B4 用于评估步长钳位与分支对齐的额外开销, B5 在 B3 基础上切换为混合精度并关闭 B4 的两项保护逻辑, B6 为 B5 + CUDA Graph。各消融级别的精确定义见第 5.3 节消融配置表。

与第三章数学基础的衔接

本章的全部CUDA工程实现可追溯至第三章建立的数学框架: Block级并行映射对应第三章第3.5节的Block/Warp/Lane三级分解; 寄存器LDL^T求解对应第三章上式的SPD矩阵分解; PaddedMat6x8的Bank冲突消除服务于第三章上式的阻尼正规矩阵高效构造; 混合精度计算路径的FP32/FP64边界划分基于第三章上式的FK运算复杂度分析和上式的Jacobian扰动精度需求; 自适应阻尼策略直接实现第三章前述公式的数学形式。这一从数学推导到硬件映射的完整链路, 为第五章的实验验证提供了可逐项量化的设计分解基础。

5 实验设计与评价方法

本章只定义实验方法, 不报告数值结果。实验的目标是把标准 UR10 批量 IK 的输入、输出、对比求解器、时间口径和消融配置统一起来, 保证第 6 章的结果可以复现、可以横向比较。

5.1 实验平台与数据集

全部实验在同一台机器上完成, 硬件和软件环境如下: NVIDIA GeForce RTX 4060 Laptop GPU、Intel Core i7-13700H、Ubuntu 22.04 LTS、ROS 2 Humble、CUDA Toolkit 12.6、GCC 11.4.0, 编译目标为 `sm_89`。

主实验对象为官方 UR10 模型, 统一使用 `tool0` 作为末端执行器定义。运动学参数由 URDF 自动解析并写入常量内存, 所有求解器读取同一模型与同一目标资产。

目标位姿资产遵循确定性可复现原则: 随机种子固定为 `seed=42`, 批量规模取 $N \in \{100, 500, 1000, 5000\}$, 初始种子统一采用 `zero_seed`。目标位姿由同一 UR10 模型采样关节角后前向生成, 保证每个目标都可达。本文始终使用 `zero_seed`, 并且不使用轨迹参数化或回放链路生成目标。

5.2 对比方法与时间口径

主对比求解器为本文 CUDA DLS 的 B5 配置, 以及 cuRobo。为提供参考, 本文还记录 `numeric_dls`、PyRoki 和 KDL 的结果, 但它们只作为 CPU 或不同框架的参考基线, 不参与主性能结论排序。

时间口径分为三层: `kernel_time_only` 只统计 GPU kernel 的设备端执行时间; `gpu_end_to_end_time` 在此基础上加上 H2D/D2H 传输和设备同步; `host_api_total_time` 进一步加上主机侧准备和 API 调用总耗时。CUDA 与 cuRobo 的主 GPU 对比采用可复现的 GPU 侧计时口径; CPU 参考求解器 (`numeric_dls`、PyRoki、KDL) 仅作为数量级参照, 不参与 GPU 主结论。`host_api_total_time` 作为工程端到端补充口径保留。

5.3 消融配置

本文采用 B0-B6 七组消融配置。B0 为 FP64 基线；B1 增加常量内存广播；B2 在 B1 基础上增加 **PaddedMat6x8**；B3 在 B2 基础上增加自适应阻尼；B4 在 B3 基础上增加步长钳位与分支对齐；B5 在 B3 基础上切换为混合精度（FP32 主体 + FP64 LDLT 关键路径）；B6 为 B5 + CUDA Graph。

级别	ConstMem	PaddedMat	Adaptive Damping	Step Clamp	Branch Align	Precision	Graph
B0	—	—	—	—	—	FP64	—
B1	✓	—	—	—	—	FP64	—
B2	✓	✓	—	—	—	FP64	—
B3	✓	✓	✓	—	—	FP64	—
B4	✓	✓	✓	✓	✓	FP64	—
B5	✓	✓	✓	—	—	FP32+FP64	—
B6	✓	✓	✓	—	—	FP32+FP64	CUDA Graph

消融实验在 **N=100/500/5000** 上测量，重复次数 **R=30**。这样安排的目的，是把内存层次、数值策略、精度路径和 launch 组织四类因素的边际贡献拆开。

5.4 性能指标

本文使用四类指标：吞吐量、收敛率、平均迭代次数和 speedup。吞吐量（Query Throughput）定义为单位时间内处理的目标总数（ N/T_{total} ），收敛率单独报告；收敛率定义为在最大迭代次数内满足误差阈值的目标比例；平均迭代次数定义为收敛目标的平均 DLS 迭代步数；speedup 定义为不同求解器吞吐量之比。

此外，本文使用 Nsight Compute 观察 kernel 的 Compute Throughput、DRAM Throughput、Registers/Thread、Achieved Occupancy、Shared Bank Conflicts 和 Kernel Duration，以判断瓶颈究竟来自计算、寄存器还是内存访问。

5.5 统计规程

每个 (**N**, **配置**) 组合先进行 3 次预热，再进行 30 次正式重复。正式结果取算术均值，并保留标准差作为测量噪声参考。PyRoki 的 JIT 预热时间不计入正式计时。

6 实验结果与性能分析

本章报告基于统一基准测试框架的实验结果，包括主 GPU 对比、逐级消融分析、Nsight Compute 架构级 profiling 以及与 cuRobo 的技术路线对比。实验设置（机器人模型、收敛阈值、时间口径、求解器分类）已在第五章详述，本章不再重复。

6.1 主 GPU 对比结果

6.1.1 吞吐量与收敛率总表

表 6.1 汇总了 CUDA B5（混合精度）与 cuRobo 在 N=100 至 N=5000 四个批量级别上的吞吐、GPU 时间与收敛率。两求解器统一采用 Medium 阈值（10 mm / 5°）。所有数据为 repeat=30、zero_seed 的算术均值。

N	Solver	Repeat	GPU时间 (ms)	Throughput targets/s	ConvRate	Avg Iters
100	cuda (B5 mixed)	30	0.890	112,414	1.000	14.47
100	curobo	30	32.08	3,118	1.000	—
500	cuda (B5 mixed)	30	3.160	158,251	0.998	15.13
500	curobo	30	31.56	15,844	1.000	—
1000	cuda (B5 mixed)	30	6.738	148,412	0.998	15.88
1000	curobo	30	31.64	31,611	1.000	—
5000	cuda (B5 mixed)	30	29.641	168,683	0.9998	15.24
5000	curobo	30	32.25	155,059	1.000	—

两求解器统一采用 Medium 阈值（ $\epsilon_p=0.01\text{ m}$, $\epsilon_R=0.0873\text{ rad} \approx 5^\circ$ ）。cuRobo 的 GPU 时间由 host 端 `time.perf_counter()` 测量（含 Python→C++→CUDA 全调用栈），CUDA B5 的 GPU 时间由 CUDA event 测量（纯 kernel 执行时间）。cuRobo 不暴露内部迭代次数，标记为 "—"。B2/B3 等消融配置的完整数据见 6.3 节消融实验，其收敛阈值与主表不同（见该节说明），不参与本节主性能排名。

6.1.2 加速比汇总

表 6.2 以 cuRobo 为基准，计算 CUDA B5（混合精度）在不同批量下的吞吐加速比。两求解器统一采用 Medium 阈值（10 mm / 5°）。

N	CUDA B5 (targets/s)	cuRobo (targets/s)	B5 vs cuRobo
100	112,414	3,118	36.1×
500	158,251	15,844	10.0×
1000	148,412	31,611	4.7×
5000	168,683	155,059	1.09×

B5 与 cuRobo 统一采用 Medium (10 mm / 5°) 阈值。B3（FP64）等消融配置与 cuRobo 的对比见 6.3 节，但其收敛阈值与主表不同，加速比数值应理解为消融分析中的参考量级而非主性能排名。

6.1.3 CPU参考基线

为量化GPU加速的量级，表 6.4 给出两个CPU求解器在相同benchmark条件下的参考数据。Orocos KDL 为工业级C++运动学库，代表成熟CPU串行实现；`numeric_dls` 为本文附带的纯Python/Numpy DLS参考实现，代表未优化的原型级CPU基线。所有数据采用 `zero_seed` 策略、repeat=30。

N	Solver	Repeat	Host时间 (ms)	Throughput targets/s	ConvRate	Avg Iters
100	KDL (C++ CPU)	30	101.4	986.1	1.000	—
100	numeric_dls (Python CPU)	30	2,177.8	45.7	0.010	36.56
500	numeric_dls (Python CPU)	30	10,322.2	48.0	0.014	30.93
1000	numeric_dls (Python CPU)	30	20,111.0	49.8	0.015	30.06
5000	numeric_dls (Python CPU)	30	81,342.2	59.8	0.015	27.36

KDL 仅在 N=100 条件下完成 repeat=30 测量——更大批量的 KDL 串行测试时间过长（N=500 预估约 8 分钟/次），未纳入正式实验计划。KDL 的 986.1 targets/s 反映的是单线程串行 IK 的吞吐上限，作为保守的 CPU 参考点。

从表 6.4 与表 6.1 的对比可得两个基本量级判断：（1）CUDA B5 相对 KDL（C++ CPU）的吞吐优势约为 114×（N=100：112,414 vs 986 targets/s），相对 numeric_dls（Python CPU）的优势约为 2,300–3,000×；（2）numeric_dls 在 zero_seed 下收敛率仅 1.0–1.5%，而同样的 DLS 算法在 CUDA 上以 B3/B4/B5 配置达到 99.8–100%——该差异并非算法本身的问题，而是 zero_seed（全零初始关节角）使大多数目标从远离解的位置出发，Python CPU 的串行迭代在 160 次上限内无法收敛。这一对比从侧面印证了批量并行对 DLS 类迭代方法的实际价值：当大量目标同时求解时，per-target 的固定迭代预算在 GPU 上可以被大量并行目标摊薄，而在 CPU 串行模式下则逐一累积。

CPU 参考基线沿用原始 30 mm / 30° 阈值（与早期单阈值实验一致），仅作数量级参照，不参与 Medium 阈值下的主结论。KDL 和 numeric_dls 未按 Medium 阈值重测。

6.1.4 结果分析

从表 6.1、表 6.2 和表 6.4 可归纳以下三个观察。

- 第一，CUDA B5 在小至中批量上表现出显著的吞吐优势。在 N=100 条件下，B5（混合精度）在 Medium 阈值（10 mm / 5°）下吞吐为 cuRobo 的 36.1 倍（112,414 vs 3,118 targets/s）；在 N=500 和 N=1000 条件下，B5 分别领先 10.0 倍和 4.7 倍。该差异主要与两类方法的 per-target 计算组织方式有关：CUDA DLS 为每个目标分配一个 block（128 线程），其固定开销在 block 粒度的调度中被有效摊销（per-target 约 8.9 μs at N=100）；而 cuRobo 作为基于 PyTorch 的框架型求解器，其跨语言调用栈（Python→C++→CUDA→C++→Python）在小批量下的固定开销占总耗时比例较高。消融实验（见 6.3 节）表明，即使在 FP64 全精度配置（B3）下，CUDA DLS 在 N=100 时的吞吐（51,361 targets/s）仍为 cuRobo 的 16.5 倍，验证了低 per-target 开销对小批量性能的决定性作用。
- 第二，混合精度（B5）在 Medium 阈值下于 N=5000 达到 cuRobo 吞吐的 1.09 倍。FP64 全精度的 B3 配置在 N=5000 时吞吐为 66,050 targets/s，约为 cuRobo 的 0.43 倍；而 B5（混合精度）以 164,207 targets/s（消融数据）或 168,683 targets/s（主表数据，因实验批次差异有小幅波动）达到 cuRobo 的 1.09 倍。这一转变来自混合精度策略的叠加：FK、Jacobian 和阻尼正规矩阵构造中 FP32 运算替代 FP64 后计算延迟降低，同时 FP32 数据宽度减半缓解了共享内存访问压力（bank conflict 从 3,522 降至 1,295，减少 49%，基于 B4→B5 的 NCU

对比)；而 LDLT 求解器、阻尼调节与收敛判定等关键路径保留 FP64，使收敛率维持在 99.98% 以上。在 Strict 阈值 (5 mm / 1°) 下，B5 仍以 1.05× 领先 cuRobo，证明该优势并非由宽松阈值拟合。

第三，两条技术路线的性能边界在混合精度引入后发生变化。 在 FP64-only 条件下 (B3)，CUDA DLS 仅在 $N \leq 1000$ 范围内领先 cuRobo，于 $N=5000$ 时落后约 2.3 倍 (66,050 vs 155,059)；引入混合精度后 (B5)，CUDA DLS 在 $N=100$ 至 $N=5000$ 的全部测试批量上均达到或超过 cuRobo 的吞吐水平。这表明在本文测试口径下，精度配置和底层实现对大批量吞吐具有决定性影响。B5 的混合精度策略——FP32 用于计算密集型但数值容忍度高的 FK/Jacobian/Hessian，FP64 用于精度敏感的 LDLT 分解与回代——在保持收敛率无损的前提下释放了显著的吞吐增益 (B3 → B5: +120–149%)。

6.1.5 种子策略敏感性

批量IK的收敛行为不仅取决于求解器设计，还受初始种子策略的影响。为评估这一因素，表 6.5 对比了两种种子策略——**zero_seed** (所有目标从零关节角 $\mathbf{q}^{(0)} = \mathbf{0}$ 出发) 与 **home_seed** (从UR10 home configuration 出发)——在CUDA默认配置 (B4, FP64) 和cuRobo上的表现。所有数据为 repeat=30 的算术均值。

N	Solver	种子策略	Throughput (targets/s)	ConvRate	Avg Iters
100	CUDA (B4)	zero_seed	38,015	1.000	13.66
100	CUDA (B4)	home_seed	26,040	1.000	18.61
100	cuRobo	zero_seed	3,059	1.000	—
100	cuRobo	home_seed	2,550	1.000	—
500	CUDA (B4)	zero_seed	45,015	1.000	14.88
500	CUDA (B4)	home_seed	32,849	0.984	18.48
500	cuRobo	zero_seed	16,659	1.000	—
500	cuRobo	home_seed	12,105	1.000	—
1000	CUDA (B4)	zero_seed	45,530	1.000	16.02
1000	CUDA (B4)	home_seed	39,009	0.991	17.86
1000	cuRobo	zero_seed	32,414	1.000	—
1000	cuRobo	home_seed	28,160	1.000	—
5000	CUDA (B4)	zero_seed	52,666	1.000	14.63
5000	CUDA (B4)	home_seed	45,274	0.990	16.39

cuRobo 的 $N=5000$ home_seed 数据因实验计划限制未采集。cuRobo 不报告 per-target 迭代次数 (其内部 particle 搜索的迭代统计口径与 DLS 不同)。

从表 6.5 可归纳三个发现。(1) **zero_seed 是更保守但也更稳定的基线。** 对 CUDA DLS 而言，zero_seed 在所有批量上均达到 100% 收敛率，而 home_seed 在 $N \geq 500$ 时收敛率轻微下降至 98.4–99.1%。zero_seed 的吞吐亦系统性地高于 home_seed (约 16–46%)，主要因为前者避免了 home configuration 附近的局部极小区域，平均迭代次数更低 (13.7–16.0 vs 16.4–18.6)。(2) **cuRobo 对种子策略不敏感。** cuRobo 的 particle 搜索机制 (每目标 200 粒子 × 多 seed) 使其在两个种子策略下均维持 100% 收敛率，吞吐差异仅 15–17%。这反映了两种求解思路的本质差异：DLS 单种子迭代对初始条件更敏感，而 particle 搜索通过多样本覆盖降低了对单

一种子的依赖。(3) **种子策略的选择不影响本文主结论**。本文主实验统一采用 `zero_seed`——它从运动学上最不利的初始条件出发，对求解器收敛能力构成最严苛的测试；在此条件下 B5 仍达到 99.98% 以上收敛率，说明本文方法的鲁棒性并非来自有利的初始种子选择。

6.2 多阈值敏感性分析

本节评估 CUDA B5 在不同精度等级下的性能退化行为。三档阈值定义见第 3 章收敛判据，此处按 Loose (30 mm / 10°)、Medium (10 mm / 5°, 主 benchmark) 和 Strict (5 mm / 1°) 三级递进。

6.2.1 三档阈值全量对比

表 6.3 汇总了 CUDA B5 与 cuRobo 在三级阈值、四个批量规模下的吞吐、收敛率与加速比。所有数据为 repeat=30、zero_seed、统一目标资产的算术均值。

N	阈值	ϵ_p/ϵ_R	CUDA B5 TP	CUDA Conv	cuRobo TP	cuRobo Conv	Speedup
100	Loose	30mm/10°	103,361	1.000	3,140	1.000	32.9×
100	Medium	10mm/5°	112,414	1.000	3,118	1.000	36.1×
100	Strict	5mm/1°	108,545	1.000	3,062	1.000	35.5×
500	Loose	30mm/10°	143,925	0.996	15,733	1.000	9.1×
500	Medium	10mm/5°	158,251	0.998	15,844	1.000	10.0×
500	Strict	5mm/1°	150,832	0.998	16,824	1.000	9.0×
1000	Loose	30mm/10°	153,620	0.998	32,867	1.000	4.7×
1000	Medium	10mm/5°	148,412	0.998	31,611	1.000	4.7×
1000	Strict	5mm/1°	145,052	0.998	33,630	1.000	4.3×
5000	Loose	30mm/10°	175,075	0.9994	157,397	1.000	1.11×
5000	Medium	10mm/5°	168,683	0.9998	155,059	1.000	1.09×
5000	Strict	5mm/1°	164,030	0.9998	155,676	1.000	1.05×
10000	Loose	30mm/10°	171,492	0.9987	42,892	1.000	4.00×
10000	Medium	10mm/5°	165,550	0.9996	41,832	1.000	3.96×
10000	Strict	5mm/1°	164,210	0.9996	44,828	1.000	3.66×

CUDA B5 数据来自 A7 二进制（混合精度），cuRobo 使用相同 target/seeds/阈值。cuRobo 在所有配置下 ConvRate 均为 1.000（其内部 particle 搜索的多样本覆盖保证收敛）；CUDA B5 收敛率在 Medium 与 Strict 下为 0.998–0.9996，与 Loose 无实质差异。N=10000 的 cuRobo 数据使用 `bench_curobo.py` 正式 benchmark 函数验证（与 N≤5000 相同调用路径），Medium 阈值下正式函数测得 TP=41,832，与内联脚本估算（TP≈44,993）量级一致。

6.2.2 阈值收紧对 CUDA B5 的影响

从 Loose 到 Strict，CUDA B5 吞吐波动在 ±6% 以内，收敛率维持在 0.998 以上。各 N 下的关键指标变化：

N	TP 变化 (Loose → Strict)	ConvRate 变化	Avg Iters 变化	PosErr (Strict)
100	+5.0%	1.000 → 1.000	13.95 → 14.86	1.6 mm
500	+4.8%	0.996 → 0.998	14.55 → 15.49	2.2 mm
1000	-5.6%	0.998 → 0.998	15.34 → 16.27	2.1 mm
5000	-6.3%	0.9994 → 0.9998	14.68 → 15.62	1.8 mm
10000	-4.2%	0.9987 → 0.9996	14.80 → 15.73	1.8 mm

平均迭代次数从 Loose 到 Strict 仅增加 6.5% (13.95 → 14.86 at N=100)，远低于 160 的最大迭代上限。该轻度增长来自更严格的姿态阈值需要 DLS 多执行 1–2 轮迭代以达到亚毫米/亚度级精度——但这一增量对总吞吐的影响被单 kernel 零同步架构摊销。

N=10000 的特殊意义。 N=10000 的数据具有独立于阈值分析的额外价值——它揭示了两种求解器在批量扩展性上的本质差异。CUDA B5 在 N 从 5000 增至 10000 时，吞吐从 168,683 轻微调整为 165,550 (-1.9%)，GPU kernel 时间从约 29.6 ms 精确翻倍至约 60.4 ms——这是 1 block/target 映射的教科书级线性扩展行为：N 翻倍意味着 block 数翻倍，在 SM 数量固定的条件下 GPU 时间精确翻倍，吞吐保持恒定。吞吐的 -1.9% 微降来自 N=10000 下 grid 规模增大导致的 SM 间 wave 调度微调，属于正常波动范围。

cuRobo 则表现出截然相反的批量扩展行为：N 从 5000 增至 10000 时，吞吐从 155,059 骤降至 41,832 (-73.0%)，单次求解时间从约 32 ms 增至约 239 ms (+646%)，远超 2× 线性增量的预期。这一非线性退化与收敛阈值无关——Loose、Medium、Strict 三档下 cuRobo N=10000 的吞吐均聚集在 42k–45k 窄区间内，表明瓶颈不在求解器计算量，而在框架层级的批量管理机制。cuRobo 作为基于 PyTorch 的 GPU 框架型求解器，其批量求解依赖 PyTorch CUDA Caching Allocator 管理 GPU workspace、Python ↔ C++ 跨语言调度以及内部 tensor 生命周期管理——这些框架层级机制在特定 batch size 下可能触发内存重分配、stream 同步或调度粒度变化，导致求解时间非线性增长。具体退化机制因 cuRobo 的部分实现细节未公开而无法从本文端完整诊断，但其 N=5000 → 10000 的实测吞吐逆降是客观的实验事实。

6.2.3 阈值鲁棒性的结构基础

CUDA B5 在三档阈值下的稳定表现源于三个架构决策的协同效应：

- (1) 单 kernel 全迭代封装。** 160 轮 DLS 迭代在单个 CUDA kernel 内完成，每增加一轮迭代的边际成本仅为该轮纯 GPU 计算时间（约 0.02 ms at N=5000），不存在 host-device 同步的放大效应。相较之下，cuRobo 的跨语言调用栈（Python → C++ → CUDA → C++ → Python）使其固定开销在 N=100 时占总耗时 90% 以上，阈值变化引起的计算增量被淹没。
- (2) FP64 LDLT 数值稳定性。** 6×6 系统在寄存器中以双精度求解。FP64 提供 15 位有效数字，即使在 Jacobian 条件数达 10⁶ 时仍能保证分解精度。DLS 求解器在 14–16 轮迭代后的自然收敛精度（Strict 下 PosErr 均值 1.8 mm, RotErr 均值远低于 1°）已大幅超越最严格的阈值要求——收敛不是因为“恰好满足阈值”，而是因为求解器本身的精度上限足够高。
- (3) 寄存器 LDLT 的固定成本。** 每轮 86 次标量运算（65 FMA, 21 DIV）在寄存器中完成，无内存访问、无 warp 内同步。每轮迭代时间在 1.945–1.951 μs 内波动 (<0.3%)。无论阻尼因子 λ 如何变化，LDLT 的计算路径和操作数均不变，不存在 LM 线搜索的步长回退风险。

综合以上三点，CUDA B5 的阈值鲁棒性并非偶然的实验结果，而是硬件感知设计的内在属性：单 kernel 架构使迭代增量成本线性可控，FP64 精度使收敛目标在求解器能力包络之内，寄存器 LDLT 使每轮成本确定。这三个

要素协同保证了即使在 Strict (5 mm / 1°) 极端条件下，CUDA B5 仍能维持 0.998+ 收敛率和 1.05× cuRobo 的加速比。

6.2.4 大规模批量线性度分析

阈值敏感性分析的一个意外收获是揭示了两类求解器在批量扩展性（batch scalability）上的结构性差异——而且这一差异的性质比此前基于 $N \leq 5000$ 数据的外推预期更为复杂和深刻。为进一步精确定位两类求解器的批量扩展行为，本文在 $N=100 \rightarrow 10000$ 范围以 1000 为固定步长进行了全量程扫描（共 12 个 N 值），所有数据使用 Medium 阈值（10 mm / 5°）、repeat=30、zero_seed 的统一口径。

全量程批量扩展对比数据（Medium 阈值 10mm/5°，步长 1000，repeat=30，zero_seed）：

N	CUDA B5 TP	CUDA GPUms	CUDA Conv	cuRobo TP	cuRobo HostMs	cuRobo 状态	加速比
100	112,414	0.89	1.000	3,118	32.1	✓	36.1×
500	158,251	3.16	0.998	15,844	31.6	✓	10.0×
1000	148,412	6.74	0.998	31,611	31.6	✓	4.7×
2000	156,007	12.82	1.000	62,455	32.0	✓	2.5×
3000	160,785	18.66	1.000	95,300	31.5	✓	1.69×
4000	166,898	23.97	0.999	18,297	218.6	●	9.1×
5000	168,683	29.64	0.9998	155,059	32.3	✓	1.09×
6000	167,030	35.92	0.9998	182,869	32.8	✓	0.91×
7000	166,933	41.93	0.9996	30,643	228.4	●	5.4×
8000	173,709	46.05	0.9999	248,732	32.2	✓	0.70×
9000	164,979	54.55	0.9999	38,126	236.1	●	4.3×
10000	165,550	60.40	0.9996	41,832	239.0	●	3.96×

✓ = 正常模式（~32ms host time）；● = 退化模式（~230ms host time，约 7× 退化）。N=4000 经两次独立复测均退化（218.6ms, 218.3ms），模式可复现。

核心发现：cuRobo 存在批量振荡而非单调悬崖。 以上全量程数据揭示了此前基于粗粒度采样（ $N=100, 500, 1000, 5000$ ）无法观测的现象：cuRobo 的求解时间在 12 个 N 值上并非单调增长或单调退化，而是在 ~32ms 的"正常模式"和 ~230ms 的"退化模式"之间来回跳变。具体而言：N=4000, 7000, 9000, 10000 四个点触发退化（HostMs=218–239ms，7.0–7.4× 正常求解时间），而 N=2000, 3000, 5000, 6000, 8000 五个点保持正常（HostMs=31–33ms）。退化不遵循"大 N 必然退化"的单调规律——N=4000 退化但 N=5000 正常，N=7000 退化但 N=8000 正常，N=8000 甚至以 248,732 targets/s 创下 cuRobo 全量程最高吞吐。这种二元状态跳变——而非渐进退化——是本文的全量程扫描揭示的最重要发现。

CUDA B5：教科书级线性扩展。 在相同的 12 个 N 值上，CUDA B5 的吞吐保持在 148k–174k 窄区间（±8%，以中位数 166k 为基准），GPU kernel 时间与 N 的线性拟合 $R^2 > 0.999$ 。每 target 计算量完全确定（FK + Jacobian + LDLT + Update 在固定迭代轮数内完成），无 block 间通信，无动态内存分配，无 GPU-CPU 同步

——工作量线性增长 → 执行时间线性增长 → 吞吐恒定。只要 GPU 有足够的全局内存容纳所有 target 的中间状态（每 target 约 2 KB， $N=10^5$ 时约 200 MB），线性扩展预计将持续。

cuRobo 振荡退化的机制分析。 为理解振荡现象的可能成因，本文对 cuRobo 的 `solve_pose()` 实现进行了源码级分析（cuRobo `curobo/_src/solver/solver_ik.py:651-683`）。关键观测如下：

```
# cuRobo solve_pose() 核心逻辑（简化）
max_batch = self.config.max_batch_size # 本文设置 = N
batch_size = goal_tool_poses.batch_size # = N
needs_pad = batch_size < max_batch      # = False (batch_size ==
max_batch)
if needs_pad:                            # 不执行
    goal_tool_poses, ... = _pad_batch_inputs(...)
batch_size = max_batch                  # = N
result = self._solve_impl(solve_state=..., ...) # 单次 GPU 调用
```

在本文配置下（`max_batch_size=N`，`batch_size=N`），cuRobo 不做内部 sub-batch 拆分——整个 batch 在单次 `_solve_impl()` 中处理。因此，230ms 的退化时间不是“多个 sub-batch 累计耗时”，而是单次 `_solve_impl()` 因 GPU 内存管理开销而被阻塞。

基于以上源码证据和实测退化特征，本文提出退化根因假设：**PyTorch CUDA Caching Allocator 的内存碎片化/重分配**。cuRobo 在初始化时为 `max_batch_size` 预分配 GPU workspace，大小约正比于 N （~40 KB/target，含 particle 状态、L-BFGS 历史、中间 tensor）。当 workspace 大小跨越 PyTorch caching allocator 的内部缓存段边界时（通常位于 128MB, 256MB, 384MB 等对齐边界），allocator 触发 `cudaFree` + `cudaMalloc` 重分配和 CUDA stream 同步，耗时约 200ms。支持此假设的证据包括：

- 退化幅度（~200ms）与 CUDA 内存重分配典型耗时一致。** 所有退化点的额外开销均约 200ms（218–32=186ms, 228–32=196ms, 236–32=204ms, 239–32=207ms），这与 `torch.cuda.empty_cache()` + 重新分配 200–400MB GPU 内存的典型耗时（150–250ms on RTX 4090）高度吻合。
- 退化点的收敛率轻微下降（1.000 → 0.9998）。** 暗示 GPU 内存重分配改变了 tensor 的内存对齐，影响了浮点运算的舍入路径，导致极个别 target 的最终迭代步产生微小偏差。
- 退化与 N 值无单调关系。** 内存碎片化取决于分配大小与缓存段边界的对齐关系，而非 N 值本身。
 $N=4000$ （~160MB workspace）可能恰好跨越 128MB 边界， $N=5000$ （~200MB）可能完全落在 128–256MB 段内， $N=7000$ （~280MB）可能恰好跨越 256MB 边界——这与观测到的振荡模式（4000 → 退化, 5000 → 正常, 7000 → 退化, 8000 → 正常, 9000 → 退化）在定性上一致。

重要声明： 本文未使用 Nsight Systems 记录 CUDA API trace，也未获取 `torch.cuda.memory_summary()` 的内存分配日志。因此，上述对退化机制的解释为基于 cuRobo 源码路径、实测退化时间特征（~200ms 固定增量、二元状态跳变、收敛率轻微伴随波动）和 PyTorch CUDA Caching Allocator 已知行为模式的**合理推断**，而非通过直接 profiling 证据确定的最终根因。精确机制的确认需要 cuRobo 维护者的协作或额外的 CUDA API 级性能追踪。

CUDA B5 的结构免疫性。 CUDA B5 对此类问题完全免疫，原因有三：(1) 每 target 固定 2KB 中间状态（裸 `cudaMalloc` 一次性分配），不存在 caching allocator 的碎片化问题；(2) 无 PyTorch 依赖——不经过任何第三方内存管理器；(3) 单 kernel 全迭代封装——kernel launch 后 GPU 完全自治，host 侧在 kernel 执行期间不进行任何内存操作。这三个设计决策使 CUDA B5 在任何 N 值下的性能行为完全可预测。

实践意义。 全量程对比数据揭示了比"cuRobo 在大批量退化"更微妙的现实：cuRobo 在某些 N 值（如 6000, 8000）实际上是两种求解器中更快的（加速比 0.91×, 0.70×），但在另一些 N 值（如 4000, 7000, 9000）因 GPU 内存管理问题而退化 5–9×。这种**不可预测的振荡**——而非简单的性能差距——才是实际工程中更棘手的问题：用户无法事先知道给定的 batch size 是否会触发退化模式，除非对所有可能的 N 值进行预扫描。CUDA B5 消除了这一不确定性：其吞吐在所有 N 值下可从事先的计算直接预测，无需任何预扫描或参数调优。

6.3 消融实验结果

6.3.1 消融级别定义

本文的 CUDA 实现包含两类设计选择：**结构性设计项**（框架核心架构特征，所有消融级别默认启用）与**可量化消融项**（可在编译时独立切换，其性能影响可直接测量）。

结构性设计项包括：（1）1 block/target、128 threads/block、四 warp 分工；（2）寄存器内 6×6 LDLT 小矩阵求解；（3）单 kernel 内完成全部 DLS 迭代。

可量化消融项的逐级配置如表 6.6 所示。

级别	ConstMem	PaddedMat	Adaptive Damping	Step Clamp	Branch Align	Precision
B0	—	—	—	—	—	FP64
B1	✓	—	—	—	—	FP64
B2	✓	✓	—	—	—	FP64
B3	✓	✓	✓	—	—	FP64
B4	✓	✓	✓	✓	✓	FP64
B5	✓	✓	✓	—	—	FP32+FP64
B6	✓	✓	✓	—	—	FP32+FP64+ Graph

B3 至 B4 增加步长钳位与分支对齐，B5 在 B3 的基础上切换为混合精度并关闭步长钳位与分支对齐，B6 在 B5 的基础上启用 CUDA Graph replay。B0–B2 追踪内存层次优化的逐步贡献，B3–B5 追踪数值策略与混合精度的叠加效应。

6.3.2 N=100（小批量，grid 未满）

Level	Throughput (targets/s)	GPU时间 (ms)	Avg Iters	Conv Rate	vs Prev
B0	7,589	13.18	31.36	0.830	—
B3	51,361	1.95	12.85	1.000	+577%
B5	113,097	0.88	14.47	1.000	+120%

B0/B3/B5 统一采用 Medium 阈值（10mm/5°）。B0 在 Medium 阈值下收敛率仅 0.830（固定 λ 无法满足更严格的精度要求），平均迭代 31.4 次/目标，其中 17% 的目标触及 max_iter=160 上限。B3 引入自适应阻尼后收敛率恢复至 1.000，迭代次数锐减至 12.85，吞吐提升 5.8 倍——这是所有消融变更中对收敛率影响最大的单一因素。B5 在 B3 基础上叠加混合精度，在保持收敛率 1.000 的同时吞吐再提升 1.2

倍，平均迭代 14.47 次。B1（常量内存）和 B2（PaddedMat 消除 bank conflict）在 N=100 条件下的吞吐增益合计 <5%（基于旧 30° 阈值数据），B4（步长钳位+分支对齐）和 B6（CUDA Graph）因引入额外开销而吞吐低于 B5。以上次要级别的边际贡献详见消融分析文本。

6.3.3 N=500（中批量）

Level	Throughput (targets/s)	GPU时间 (ms)	Avg Iters	Conv Rate	vs Prev
B0	9,896	50.52	79.74	0.522	—
B3	62,384	8.01	13.78	1.000	+530%
B5	155,071	3.22	15.13	0.998	+149%

B0/B3/B5 统一采用 Medium 阈值（10mm/5°）。B4（步长钳位+分支对齐）在 N=500 条件下吞吐较 B3 下降约 15%（旧阈值数据），因其增加的边界检查构成纯计算开销而未提供收敛收益；B5 关闭这两项后吞吐恢复并超越。B1/B2 在 N≥500 条件下的吞吐收益 <1%，数据未单独列出。

6.3.4 N=5000（大批量）

Level	Throughput (targets/s)	GPU时间 (ms)	Avg Iters	Conv Rate	vs Prev
B0	12,507	399.79	73.04	0.564	—
B3	66,050	75.70	13.69	1.000	+428%
B5	164,207	30.45	15.24	0.9998	+149%

B0/B3/B5 统一采用 Medium 阈值（10mm/5°），repeat=30 算术均值。B0 在 Medium 阈值 + N=5000 条件下收敛率仅 0.564（超过 43% 目标未能在 160 次迭代内收敛），充分说明固定 λ 策略在严格精度要求 + 大批量场景下的脆弱性。B3 的自适应阻尼将收敛率恢复至 1.000，迭代次数从 73.0 降至 13.7，吞吐提升 4.3 倍。B5 叠加混合精度后吞吐再提升 1.5 倍至 164k targets/s，GPU 时间仅 30.5 ms。B1/B2/B4/B6 的边际贡献参考消融分析文本（基于旧 30° 阈值或部分 N 值数据）。

6.3.5 消融分析

以下分析以 Medium 阈值（10mm/5°）下 B0、B3、B5 的实测数据为一级证据，B1、B2、B4、B6 的旧 30° 阈值数据为二级参考（仅用于评估边际贡献方向与幅度，不参与主性能排名）。

（一）Medium 阈值暴露了固定 λ 的脆弱性——此前被宽松阈值掩盖。在旧 30 mm/30° 阈值下，固定 λ 的 B0 配置在 N=100 时吞吐达 134k targets/s、收敛率 1.000、平均迭代仅 4.3 次——表现与自适应阻尼的 B3 接近，甚至因无分支开销而更快。然而切换到 Medium 阈值（10mm/5°）后真相暴露：B0 的收敛率在 N=100 时骤降至 0.830，N=500 时进一步跌至 0.522，N=5000 时仅 0.564——超过 43% 的目标无法在 160 次迭代内收敛。平均迭代次数从 4.3（旧阈值）激增至 31.4→79.7→73.0（新阈值），吞吐从 134k 暴跌至 7.6k→9.9k→12.5k。这一剧烈退化表明：固定 λ DLS 求解器在机器人 IK 中的"良好表现"高度依赖于宽松的收敛判据；一旦收敛判据收紧至工程合理的精度水平，其收敛能力便迅速坍塌。此前的消融分析（基于 30° 阈值）未能揭示这一关键事实。

（二）自适应阻尼的收敛恢复效应——全批量范围的正收益。从 B0 到 B3 的唯一变更是引入自适应阻尼策略（迭代 0 距离初始化、迭代 1+ Marquardt 误差驱动更新、停滞超驰）。Medium 阈值下，该变更是所有可量化消融项中对收敛率影响最显著的单一因素：

N	B0 Conv	B3 Conv	B0 Iters	B3 Iters	B0 TP	B3 TP	提升
100	0.830	1.000	31.36	12.85	7,589	51,361	+577%
500	0.522	1.000	79.74	13.78	9,896	62,384	+530%
5000	0.564	1.000	73.04	13.69	12,507	66,050	+428%

与旧阈值下的观察（N=100 时 B3 较 B2 吞吐下降 62.4%）截然相反，Medium 阈值下自适应阻尼在**全部**测试批量上均提供 4×–6× 的正向吞吐提升。旧阈值下的"负收益"实为伪象——当收敛判据足够宽松以至于固定 λ 也能快速满足时，自适应阻尼的分支逻辑（if-else、sqrt、fmin/fmax）确实构成了纯开销；但一旦收敛判据收紧至工程合理水平，自适应阻尼对迭代次数的大幅削减（减少 59%–83%）完全碾压了分支开销。**这一发现具有方法论意义：消融实验中收敛阈值的选择直接影响优化项的性能归因——过于宽松的阈值会系统性地低估收敛策略的价值。**

（三）混合精度的吞吐倍增——在收敛率无损前提下的最大化加速。 从 B3（FP64 全精度 + 自适应阻尼）到 B5（FP32 主体 + FP64 LDLT 关键路径 + 自适应阻尼），混合精度在全部测试批量上带来了 120–149% 的额外吞吐提升，是所有消融级别中对吞吐幅度最大的单一变更：

N	B3 TP	B5 TP	提升	B5 Conv
100	51,361	113,097	+120%	1.000
500	62,384	155,071	+149%	0.998
5000	66,050	164,207	+149%	0.9998

该提升来自 FK、Jacobian 和阻尼正规矩阵构造中 FP64 运算量的大幅减少，以及 FP32 数据宽度减半带来的共享内存访问压力缓解（bank conflict 从 3,522 降至 1,295，减少 49%，基于 B4→B5 的 NCU 对比）。同时，LDLT 求解器、阻尼调节和收敛判定保留 FP64 精度路径，使得 B5 的收敛率维持在 99.8% 以上（N=5000 为 0.9998，N=500 为 0.998，N=100 为 1.000），未因主体精度的降低而产生可测量的收敛退化。从 NCU 指标看，混合精度引入的类型转换（FP32→FP64 于 LDLT 求解入口，FP64→FP32 于结果回写）没有表现为主导瓶颈；主要瓶颈仍为计算吞吐与寄存器压力。

（四）B1/B2/B4/B6 的边际贡献——基于旧阈值数据的参考评估。 以下四项消融变更未在 Medium 阈值下重新测量，其边际贡献的定量评估基于旧 30° 阈值数据，方向性结论在 Medium 阈值下应仍然成立：

- **B1（常量内存）与 B2（PaddedMat 消除 bank conflict）：**在 N=100 条件下合计贡献 <5% 吞吐提升。UR10 的 FK 工作集约 912 bytes，即使不经优化的常量内存路径也能被 L1 缓存高效容纳。在大批量（N≥500）上收益可忽略。
- **B4（步长钳位 + 分支对齐）：**在所有测试批量上 B4 吞吐均低于 B3（-15% 至 -20%），收敛率不变（1.000）。0.35 rad 的步长限制对 UR10 关节空间偏保守，增加的边界检查和 wrap 逻辑构成纯计算开销。因此 B5 关闭了这两项优化。
- **B6（CUDA Graph replay）：**N=100 条件下 B6 吞吐较 B5 提升约 3.7%，主要反映 launch/replay 机制变化；event-based 计时显示 `cudaGraphLaunch` 的 kernel 执行时间与直接 launch 在测量噪声范围内一致（差异 <0.2%），表明当前 kernel 配置下 launch overhead 不是性能瓶颈。

综合消融结论，主求解器配置采用 B5（混合精度 + 自适应阻尼，关闭步长钳位与分支对齐），在 Medium 阈值（10mm/5°）下收敛率不低于 99.8%，N=100–5000 范围内吞吐 51k–164k targets/s，均达到实验中的最高或接近最高的吞吐水平。

6.4 Nsight Compute 架构级分析

本节以 N=100 为代表样例，利用 NVIDIA Nsight Compute 对 kernel 内部瓶颈结构进行定量分析。N=5000 下的 SM 填充、wave 数与调度行为与 N=100 存在差异（主要体现为 grid 饱和度），但 kernel 内部的指令 mix 与内存访问模式基本保持一致，因此 N=100 的 profiling 结论对理解整体瓶颈具有代表性。以下分析均基于此前提。

6.4.1 B4（FP64 全精度）性能画像

表 6.7 给出了 B4 配置在 N=100 条件下的 NCU 关键指标。

指标	值	解读
Compute throughput	66.89%	计算为主瓶颈
DRAM throughput	1.56%	非带宽受限
Registers/thread	94	限制 occupancy 的主因
Achieved occupancy	32.51%	受寄存器上限 × waves/SM 不足
L1/TEX hit rate	99.13%	数据复用良好
Shared bank conflicts	3,522	存在共享访问冲突
Local spilling	0	无寄存器溢出

6.4.2 B5（混合精度）性能画像

表 6.8 给出了 B5 配置在 N=100 条件下的 NCU 关键指标。

指标	值	解读
Compute throughput	60.73%	仍以计算为主
DRAM throughput	1.16%	非带宽受限
Registers/thread	98	略高于 B4（FP32 → FP64 转换逻辑）
Achieved occupancy	33.3%	与 B4 相近
L1/TEX hit rate	98.62%	仍保持高命中率
Shared bank conflicts	1,295	较 B4 减少 49%
Kernel duration	827 μs	较 B4 2,920 μs 缩短 72%

6.4.3 架构级分析

基于表 6.7 与表 6.8 的定量数据，归纳以下四项架构级结论。

第一，kernel 为计算边界而非内存边界。 B4 与 B5 的 DRAM throughput 分别为 1.56% 与 1.16%，均远低于典型的带宽受限阈值（约 60%）。两版本的 Compute throughput 分别为 66.89% 与 60.73%，表明 SM 计算管线为主要瓶颈。L1/TEX hit rate 均高于 98%，说明数据工作集在缓存中得到充分复用，与 6.2 节中内存层次优化收益有限的观察一致。

第二，寄存器压力为 occupancy 的主要限制因素。 B4 的 94 寄存器/线程与 B5 的 98 寄存器/线程，在 128 线程/block 配置下将理论 occupancy 限制至约 33%。N=100 时每 SM 仅驻留约 0.83 个 wave，进一步降低了 achieved occupancy（32.51--33.3%）。B5 较 B4 增加 4 个寄存器/线程，主要来自 FP32 → FP64 转换所需的中间变量，但该增量未对 occupancy 产生实质影响。

第三，混合精度有效缓解共享内存访问冲突。 B5 的 shared bank conflicts 为 1,295 次，较 B4 的 3,522 次减少 49%。该改善源于 FP32 数据宽度为 FP64 的一半，对共享内存的自然对齐访问减少了两路和三路 bank 冲突的发生频率。然而，即使以 B4 的峰值 3,522 次冲突计，其占总共享访问波前的比例不足 2%——即使完全消除，预期加速也在测量噪声范围之内。因此，bank conflict 的减少是混合精度加速的伴随现象，而非加速的主因。

第四，FP64 计算管线在混合精度模式下仍保持高利用率。 B5 的 FP64 pipeline 利用率达 60.73%，与 B4 的 66.89% 处于同一量级。这表明尽管 FK、Jacobian 和阻尼正规矩阵的主体计算已切换为 FP32，LDLT 求解器的 FP64 路径仍为 SM 提供了充足的计算负载。从当前 NCU 指标看，混合精度引入的类型转换（FP32 → FP64 于 LDLT 求解入口，FP64 → FP32 于步长计算回写）没有表现为主导瓶颈；主要瓶颈仍为计算吞吐与寄存器压力。

B5 的 kernel 执行时间从 B4 的 2,920 μ s 降至 827 μ s（缩短 72%），该加速来自 FP32 计算延迟降低、共享内存带宽利用率提升以及 bank conflict 减少三者的叠加效应，其中计算延迟降低贡献最大。

6.5 与 cuRobo 的性能边界讨论

本文与 cuRobo 的比较是统一模型（官方 UR10）、统一 TCP（tool0）、统一 target/seeds 资产、统一收敛阈值和统一时间口径下的任务级对比，而非等计算量、等精度、等 seed 数或等优化器结构下的对照实验。两种方法代表 GPU IK 求解的两类并行化路线，其性能特征在不同批量规模下呈现差异化的适用区间。

6.5.1 技术路线差异

表 6.9 从求解器、精度、每目标映射方式、并行策略和批处理范围五个维度对比了两条路线的设计选择。

维度	本文 (CUDA DLS)	cuRobo
求解器	Damped Least Squares (DLS)	L-BFGS + particle 搜索
精度	混合精度 (FP32 主体 + FP64 LDLT)	FP32
每目标映射	1 block × 128 threads	多 particle × 多 seed
并行策略	轻量级单 kernel	批量 particle 搜索
批处理范围	1–5000+	100–5000+

cuRobo 的核心设计在算法层面——通过 L-BFGS 优化器与 particle 搜索的组合，利用多粒子并行的方式填充 GPU 的计算单元，在较大批量下实现高 SM 利用率。本文的核心设计在 GPU 硬件适配层面——通过线程到 warp 的精确分工、混合精度调度、共享内存布局优化、寄存器级 6×6 LDLT 求解器与常量内存广播，降低每个目标的固定映射开销。

6.5.2 批量规模与性能边界

为完整刻画两条路线的批量扩展特征，本文在 N=100 → 10000 范围以 1000 为步长进行了全量程扫描（12 个 N 值）。表 6.10 汇总了所有数据点。

表 6.10：全量程批量规模 vs 吞吐（Medium 阈值 10 mm / 5°，步长 1000）

N	CUDA B5 (targets/s)	cuRobo (targets/s)	比值 (B5/cuRobo)	cuRobo 状态
100	112,414	3,118	36.1×	✓
500	158,251	15,844	10.0×	✓
1000	148,412	31,611	4.7×	✓
2000	156,007	62,455	2.5×	✓
3000	160,785	95,300	1.69×	✓
4000	166,898	18,297	9.1×	●
5000	168,683	155,059	1.09×	✓
6000	167,030	182,869	0.91×	✓
7000	166,933	30,643	5.4×	●
8000	173,709	248,732	0.70×	✓
9000	164,979	38,126	4.3×	●
10000	165,550	41,832	3.96×	●

✓ = 正常模式 (~32ms host time); ● = 退化模式 (~230ms host time)。全量程数据见 [data/全量程对比/full_range_comparison.csv](#)。

从表 6.10 可以识别出五条规律。

第一，CUDA DLS 的 per-target 固定开销显著低于 cuRobo。 在 N=100 端，CUDA B5 的吞吐为 cuRobo 的 36.1 倍。该差异来源于两种映射方式的本质不同：CUDA DLS 每个目标的固定开销仅为 1 个 block 的调度与 128 线程的协作（约 0.89 ms 分摊至 100 目标后，per-target 约 8.9 μ s），而 cuRobo 的 particle 搜索为每个目标分配 200 粒子 $\times$ 多 seed 的搜索预算，其 per-target 开销不会因 batch size 减小而等比降低。

第二，cuRobo 在正常模式下具有可观的批量弹性，但该弹性被不可预测的退化模式打断。 在 8 个正常 N 值上，cuRobo 的吞吐随 N 从 3,118 (N=100) 单调增长至 248,732 (N=8000，约 79.8 倍)，表现出了 particle 搜索架构在 SM 利用率上的批量弹性优势——N=6000 和 N=8000 时 cuRobo 甚至分别以 1.10 $\times$ 和 1.43 $\times$ 的比率快于 CUDA B5。然而，在 4 个退化 N 值（4000, 7000, 9000, 10000）上，cuRobo 的吞吐骤降至 18k–42k 区间——不仅远低于正常模式，甚至低于 N=500 时的水平（15,844）。这种“正常模式越来越快，退化模式回到原点”的振荡模式，使得 cuRobo 的吞吐曲线在 N=100 $\rightarrow$ 10000 范围内呈现锯齿状，而非单调递增。

第三，CUDA B5 的吞吐与 N 无关（批量无关性），而 cuRobo 的吞吐与 N 的奇偶性无关但与 GPU 内存分配边界有关。 CUDA B5 在 12 个 N 值上的吞吐 148k–174k ($\pm 8\%$)，GPU 时间与 N 严格线性 ($R^2 > 0.999$)。这一批量无关性来自 1 block/target 映射的结构确定性。相比之下，cuRobo 的吞吐波动范围达 0.70 $\times$ –36.1 $\times$ （最高/最低 = 79.8 $\times$ ），且波动模式不由 N 值本身决定（N=8000 正常但 N=9000 退化，N=5000 正常但 N=4000 退化）。

第四，加速比曲线呈振荡 U 型而非单调 V 型。 在正常 N 值序列（100 $\rightarrow$ 2000 $\rightarrow$ 3000 $\rightarrow$ 5000 $\rightarrow$ 6000 $\rightarrow$ 8000）上，B5/cuRobo 加速比从 36.1 $\times$ 单调递减至 0.70 $\times$ （即 cuRobo 反超 1.43 $\times$ ）。但在退化 N 值（4000, 7000, 9000, 10000）上，加速比跳升至 3.96 $\times$ –9.1 $\times$ 。从 N=100 $\rightarrow$ 10000 的整体视角看，两条路线的性能关系应被描述为“小批量碾压（36 $\times$ ） $\rightarrow$ 中批量接近乃至被反超（0.70 $\times$ ） $\rightarrow$ 大批量因对方退化而再次拉开（3.96 $\times$ ）”——一升一降间，不是“趋同”或“交叉”，而是**振荡共存**。

第五（新增），振荡本身——而非性能差距——是 cuRobo 在实际部署中的主要风险。 在 cuRobo 正常模式下，其性能在 $N \geq 5000$ 时实际上优于 CUDA B5（最高 $1.43\times$ ）。如果 cuRobo 在所有 N 值上均能保持正常模式，它将是大批量场景中更优的选择。然而，退化模式的存在使得 cuRobo 的性能具有二元不确定性：用户在提交一个 batch 之前无法预知这次求解将耗时 32ms 还是 230ms。这是因为退化并非由用户可控的参数（如 N 值大小、阈值松紧）决定，而是由 GPU 内存管理器的内部状态决定——这是一个对用户完全透明的变量。

6.5.3 路线选择的启示

上述全量程对比为 GPU IK 求解器的设计提供了以下启示。

（1）per-target 开销决定小批量性能。 对于单次或小批量 IK 查询场景（ $N \leq 1000$ ），低 per-target 固定开销的求解器具有显著优势。CUDA DLS 的 1 block/target 映射在 $N=100$ 条件下提供了 36.1 倍的吞吐优势。

（2）性能可预测性是大批量场景的关键需求。 cuRobo 的正常模式在大批量下（ $N \geq 5000$ ）实际上提供了比 CUDA B5 更高的吞吐（最高 $1.43\times$ at $N=8000$ ）。然而，其退化模式的不可预测性——你不知道下一次求解是 32ms 还是 230ms——对需要稳定延迟的上层系统（如机器人控制回路）构成实质性风险。CUDA B5 虽然在大批量正常模式下略慢于 cuRobo（ $0.70\times-0.91\times$ ），但其吞吐在任何 N 值下均可从事先的计算精确预测，无需预扫描或运行时试探。对于离线批量求解场景，若稳定性优先于峰值性能，CUDA B5 是更可靠的选择；若峰值性能优先且允许对每个 batch size 进行预测试，cuRobo 的正常模式在 $N \geq 5000$ 时具有吞吐优势。

（3）GPU 内存管理应被视为求解器性能的独立评估维度。 cuRobo 的退化模式——一种可能机制为 PyTorch CUDA Caching Allocator 在特定分配大小下的碎片化行为（详见 6.2.4 节的假设与声明）——具有超出本文 benchmark 的方法论意义。它表明，在评估基于 PyTorch 等高层框架的 GPU 求解器时，“在 5 个典型 N 值上测量平均吞吐”可能漏掉关键的异常点。建议 GPU IK benchmark 采用细粒度全量程扫描（步长 ≤ 1000 ）以捕获此类振荡行为。

综合以上分析，本文的 CUDA DLS 路线在 $N=100-10000$ 的全量程批量范围内提供了**可预测**的吞吐水平——虽然在大批量正常模式下略慢于 cuRobo（ $0.70\times-0.91\times$ ），但其吞吐的批量无关性（ $\pm 8\%$ 波动）和 GPU 时间的严格线性（ $R^2 > 0.999$ ）使其成为两种求解器中唯一具有完全确定性性能行为的方案。对更大批量（ $N > 10^4$ ）、多 seed 策略以及 7-DOF 冗余机械臂的性能边界，仍需进一步的实验验证。

7 扩展性验证

本章只验证框架的正确性和参数化可迁移性，不做正式性能对比。验证对象为 Franka Panda 7DOF，用来说明本文的 6DOF CUDA 实现是否可以在不改变核心映射结构的前提下扩展到冗余自由度机械臂。

7.1 参数化迁移

从 UR10（6DOF）到 Panda（7DOF）的改动主要是维度常数替换：**DOF** 从 6 改为 7，Jacobian 从 6×6 变成 6×7 ，阻尼正规矩阵从 6×6 变成 7×7 ，LDLT 由 6×6 扩展到 7×7 ，FK 链段数从 6 增至 7。线程映射、warp 分工、共享内存组织和自适应阻尼的控制逻辑保持不变。

7.2 正确性验证

验证只检查数值正确性，不检查吞吐。结果表明：CUDA FK 与 CPU 参考在机器精度量级上一致，最大逐元素误差为 2.78×10^{-16} ；在 $N=10$ 个随机目标上，CUDA DLS IK 的收敛率为 **50% (5/10)**，与 Python CPU 参考完全一致。这说明 7DOF 迁移后，核心数值路径没有被破坏。

7.3 本章结论

Panda 验证的作用是证明本文的 CUDA 映射并不依赖于“恰好是 6 自由度”这一事实，但它不是正式 benchmark，也不与 cuRobo 做性能排名。7DOF 的完整性能测试留给后续工作。

8 讨论

8.1 范围边界

本文的研究对象严格限定为标准 UR10 批量 IK 查询。它关注的是独立目标位姿的批处理求解，不涉及路径规划、避障 IK、轨迹优化或其他更外层的问题。这样做的目的是让性能结论足够清晰，不把不同问题类型混在一起。

8.2 设计取舍

实验结果说明，内存层次优化只带来有限收益，自适应阻尼决定收敛率，混合精度决定吞吐，而寄存器压力和共享内存访问决定了 kernel 的硬件边界。换句话说，本文的性能提升不是来自某个单点技巧，而是来自把数学形式和硬件映射对齐。

8.3 局限性

本文的局限性主要有三点。第一，7DOF 只做了正确性验证，没有做完整性能 benchmark。第二，当前实现仍然使用 FP64 LDLT 关键路径，寄存器压力较高。第三，当前主线只覆盖单目标单 seed 的批量 IK 查询，没有探索更复杂的多 seed 组织方式。

8.4 后续工作

后续工作可以沿四个方向展开：一是补齐 Panda 7DOF 的完整 benchmark；二是继续优化寄存器压力和 occupancy；三是尝试 FP32 LDLT 加 iterative refinement；四是在严格保持 benchmark 口径的前提下，评估多 seed 和更大 batch 的调度策略。

9 结论

本文研究范围严格限定为标准 UR10 批量 IK 查询，不提出新的 IK 数学算法，也不涉及路径规划、碰撞检测、轨迹优化等更上层运动规划环节的评价。全文以同一 UR10 模型、同一 `tool0` 定义、同一批目标位姿与初始种子、统一误差阈值和时间口径为前提，确保性能比较建立在一致的基础上。

方法上，本文从 DLS 正规方程出发，将每次迭代归结为固定规模的 6×6 阻尼正规矩阵线性系统。CUDA 实现采用 1 block/target、128 threads/block 的并行映射，block 内按 4 warp 分工完成 FK/误差、数值 Jacobian、阻尼正规矩阵构造和 LDLT 求解；共享内存使用 PaddedMat6x8 布局以缓解 Bank 冲突，核心线性求解采用寄存器级 FP64 LDLT，而 FK 和 Jacobian 等主体计算运行在 FP32 路径上以降低计算成本。

实验结果表明：B0-B6 消融在 Medium 阈值 (10mm/5°) 下证实：内存层次优化 (B1 常量内存、B2 PaddedMat6x8) 的收益有限 (合计 <5%)，自适应阻尼 (B3) 在固定 λ 的 B0 基础上提供 4–6× 的收敛驱动吞吐提升——是所有消融变更中对收敛率影响最大的单一因素，混合精度 (B5) 在 B3 基础上再提供 120–149% 的吞吐提升且收敛率无损 (≥ 0.998)；三档组合阈值扫描 (Loose 30 mm/10° → Medium → Strict 5 mm/1°) 表明 CUDA B5 吞吐波动仅 $\pm 6\%$ ，收敛率维持 0.998+，即使在 Strict 极端条件下仍保持 1.05× cuRobo 的加速比——这一阈值鲁棒性来源于单 kernel 零同步架构、FP64 LDLT 数值稳定性和寄存器固定成本的协同效应；N=100 → 10000 全量程扫描 (12 个 N 值，步长 1000) 揭示了两类求解器的系统性差异：CUDA B5 吞吐保持 148k–174k ($\pm 8\%$)，GPU 时间与 N 严格线性 ($R^2 > 0.999$)，而 cuRobo 在 4/12 个 N 值上触发 ~230ms 退化模式 (7× 正常求解时间)，呈现“正常 → 退化 → 恢复 → 再退化”的批量振荡——N=10000 时 B5/cuRobo 加速比达 3.96×，修正了此前基于 N≤5000 数据的“趋同”外推，揭示了本文专用 CUDA kernel 相比高层框架型 GPU IK 实

现在批量扩展确定性与性能可预测性上的结构性优势；Nsight Compute profiling 证明 kernel 主要受计算吞吐与寄存器压力约束，而非 DRAM 带宽约束；7DOF Panda 的正确性验证表明该 CUDA 结构可保持正确收敛后迁移至不同自由度平台。

本文的局限性包括：仅以 UR10（6DOF）为完整 benchmark 主平台，Panda（7DOF）仅完成正确性验证而未建立同等规模的性能基准；cuRobo 对比是基于统一模型、目标、种子和阈值的任务级比较，不等同于等计算量、等优化器结构的组件级比较；cuRobo $N=10000$ 退化机制的分析基于公开文献和实测 scaling 行为的合理推断，cuRobo 的内部实现对本文为黑盒，其开发者在统一 benchmark 框架下通过参数调优可能获得更优的 $N=10000$ 性能；当前实现仍保留 FP64 LDLT 关键路径，寄存器压力较高。后续工作将围绕多 seed 组织方式、7DOF 完整 benchmark、更大批量 ($N>10^4$) 性能边界、寄存器压力优化以及 FP32 LDLT 加 iterative refinement 等方向展开。

参考文献

1. Universal Robots. [Universal_Robots_ROS2_Description](#).
https://github.com/UniversalRobots/Universal_Robots_ROS2_Description
 2. Sundaralingam, B., Hari, S. K. S., Fishman, A., Garrett, C., Van Wyk, K., Blukis, V., Millane, A., Oleynikova, H., Handa, A., Ramos, F., Ratliff, N., Fox, D. [cuRobo: Parallelized Collision-Free Minimum-Jerk Robot Motion Generation](#). <https://arxiv.org/abs/2310.17274>
 3. NVIDIA / NVLabs. [curobo](#). <https://github.com/NVlabs/curobo>
 4. Kim, C. M., Yi, B., Choi, H., Ma, Y., Goldberg, K., Kanazawa, A. [PyRoki: A Modular Toolkit for Robot Kinematic Optimization](#).
https://autolab.berkeley.edu/assets/publications/media/iros_25_pyroki_camera_ready.pdf
 5. PyRoki project page. <https://chungmin99.github.io/pyroki/>
 6. PyRoki source repository. <https://github.com/chungmin99/pyroki>
 7. Orocos Kinematics and Dynamics Library. https://github.com/orocos/orocos_kinematics_dynamics
 8. Orocos KDL overview. <https://docs.orocos.org/kdl/overview.html>
 9. NVIDIA Nsight Compute documentation. <https://docs.nvidia.com/nsight-compute/NsightCompute/index.html>
 10. NVIDIA Nsight Compute Profiling Guide. <https://docs.nvidia.com/nsight-compute/ProfilingGuide/index.html>
-